台灣自編大學教科書

地圖建構與同步定位 (SLAM)

Simultaneous Localization and Mapping



大學教科書

第九篇

第32–34章

AI 與語意導航

李世淵 (機器人叫獸)

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第九篇　AI、VLM 與語意導航 SLAM

第 32 章

語意 SLAM：讓地圖帶有意義

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程　/　高中 AI 機器人社團課程

學習目標

走過三十一章，同學已經能讓機器人回答「我在哪裡、地圖長什麼樣」。本章要補上第三個問題：「這個地方是什麼」。我們從傳統 SLAM 的侷限談起，說明 AI 視覺模型在 SLAM 中的分工、語意地圖的四種表示法，再處理動態物體與語意代價圖兩個實務難題，最後回顧國際研究地圖。讀完本章，同學應能：

1. 說明幾何地圖的侷限，以及機器人智慧的三個層次為何是疊加而非取代。
2. 區分偵測、分割、深度、描述、場景理解五類 AI 視覺模型在 SLAM 中的分工。
3. 說明語意地圖的四個層次，以及開放詞彙與場景圖為何是近年的研究重點。
4. 說明動態物體為何會汙染地圖，並能設計多趟一致性的剔除策略。
5. 說明語意如何進入代價地圖並改變導航決策，並能估算其代價與效益。
6. 說明語意標註的誤差如何經由外參與定位傳播到地圖上。
7. 舉例說明國際上語意 SLAM 的主要研究路線與代表系統。

走過三十一章，同學已經能讓機器人回答「我在哪裡、地圖長什麼樣」。但這還不夠——真正實用的機器人，還必須回答「我看到了什麼、這個地方代表什麼、下一步該怎麼做」。本章是第九篇的第一章，也是通往下一個十年的入口。學完本章之後，同學應能夠達成下列目標：

1. 說明傳統 SLAM 的限制，以及 AI 為何成為必要的補充。
2. 說明 YOLO、SAM、Depth Anything、DINO、CLIP 在機器人感知中的分工。
3. 說明從 Visual SLAM 到 Semantic SLAM 再到具身 AI 導航的演進脈絡。
4. 規劃機器人訓練資料的蒐集與標註流程，並訓練校園場景辨識模型。
5. 說明 VLM 如何理解自然語言導航指令，以及 RAG 如何提供校園空間知識。
6. 說明邊緣 AI 部署的關鍵技術：量化、蒸餾與記憶體管理。

7. 分析 AI 輔助 SLAM 的風險，並展望 SLAM 與基礎模型結合的未來。

核心概念

名詞	意義	關聯章節
語意地圖	在幾何地圖上附加類別、實例與關係資訊	32.3
實例分割	把影像切成一個個獨立物件，而非只有類別	32.2
開放詞彙	類別不必事先定義在固定清單中	32.3、32.6
三維場景圖	以「物件—區域—樓層」階層與關係描述空間	32.3
動態物體剔除	把會移動的東西排除在地圖之外	32.4
語意代價地圖	把「那是什麼」轉換成導航成本	32.5
標註傳播誤差	語意正確但外參或定位不準，標籤仍會貼歪	32.5、第 10 章
幾何—語意—任務	機器人智慧的三個層次，下層支撐上層	32.1

中文名詞	英文名詞	概念說明
語意 SLAM	Semantic SLAM	在幾何地圖上加註物件與場景意義的 SLAM。
具身智慧	Embodied AI	透過與實體環境互動來學習與行動的人工智慧。
開放詞彙	Open-Vocabulary	不受預設類別限制，能辨識訓練時未見過的物件。
物件目標導航	ObjectNav	接受「去找某個物體」指令的導航任務。
場景圖	Scene Graph	以節點與關係描述場景中物件及其空間關聯的結構。
視覺語言模型	VLM	能同時理解影像與語言的多模態模型。
視覺語言動作模型	VLA	直接由影像與語言產生動作指令的模型。

中文名詞	英文名詞	概念說明
檢索增強生成	RAG	先檢索相關資料再生成回答，以提升正確性。
模型蒸餾	Distillation	把大模型的能力轉移到小模型上的訓練方法。
量化	Quantization	以較少位元表示模型參數以降低記憶體與運算。
邊緣部署	Edge Deployment	在機器人本體的運算平台上執行 AI 推論。
幻覺	Hallucination	AI 產生看似合理但實際錯誤內容的現象。
世界模型	World Model	學習環境動態、能預測未來狀態的模型。

32.1　為什麼傳統 SLAM 還需要 AI

本書前三十章建立的是一套極為紮實的幾何系統。它能精確地告訴你機器人在哪、環境的形狀如何——但它有一個根本的盲點。

傳統 SLAM 知道形狀，不知道意義

一台跑著 Cartographer 的機器人，面對一扇門時，它「知道」的是：前方 1.2 公尺處有一段寬 0.9 公尺的空隙，兩側是連續的平面。它不知道那是「門」——不知道門可以通過、門可能會關上、門後面通常是另一個房間。

這個差別在簡單任務中無關緊要（避開障礙不需要知道障礙是什麼），但在真實的服務任務中卻致命：

- 使用者說「去教務處」，而地圖上只有座標，沒有名字。
- 地上有一攤水，幾何上它是平坦的地面，語意上機器人應該繞開。
- 前方是玻璃門，光達看不到（第 8 章），但視覺能辨識出門框與把手。
- 走廊上站著一個人，幾何上是障礙物，語意上機器人應該減速並禮讓，而非把他當成牆壁繞過去。

三個層次的落差

層次	回答的問題	技術	本書章節
幾何	我在哪？環境形狀如何？	SLAM	第 1–30 章
語意	我看到了什麼？這裡是什麼地方？	物件偵測、分割、VLM	本章
任務	下一步該怎麼做？	LLM Agent、VLA	本章

表 32-1　機器人智慧的三個層次

這三層是疊加而非取代的關係——這是本章最重要的觀念。語意層若沒有穩固的幾何層支撐，機器人會「知道那是門，卻走不到門口」；而幾何層若沒有語意層，機器人只能執行座標指令，無法與人自然互動。

AI 帶來什麼

1. 感知的擴充：辨識物件、分割場景、估計深度——把像素轉成有意義的資訊。
2. 語言的介面：讓使用者以自然語言下達任務，而非輸入座標。
3. 常識的推理：「洗手台通常在廁所或茶水間」——這類知識無法從幾何地圖推導，卻能大幅提升探索效率（第 24.5 節）。
4. 泛化的能力：面對訓練時未見過的環境與物件，仍能做出合理判斷。

32.2　AI 視覺模型在 SLAM 中的角色

第 23.4、24.5 節介紹過 YOLO 與 VLM，本節系統性地整理各類視覺模型的分工——這是規劃 AI 感知系統的地圖。

五類模型的分工

近年的整理相當清楚地指出各家族的專長與弱點：影像文字對比模型（如 CLIP、SigLIP）是視覺語言模型的預設前端，擅長語意檢索但在稠密預測上較弱；自監督視覺變換器（如 DINOv2、DINOv3）因訓練目標保留了細緻的空間結構，在分割、深度、對應等稠密預測上表現優異，但其特徵未與語言對齊；分割基礎模型（如 SAM、SAM 2）是物件提議與追蹤的專家；稠密預測專家（如 Depth Anything、MiDaS）則處理深度。

模型類型	代表	擅長	在 SLAM 中的角色
物件偵測	YOLO	即時框出物件類別	語意標註、動態物體剔除
影像文字對比	CLIP、SigLIP	開放詞彙的語意檢索	地點辨識、語言查詢

模型類型	代表	擅長	在 SLAM 中的角色
自監督視覺	DINOv2 / v3	稠密預測、空間結構	特徵提取、分割
分割基礎模型	SAM、SAM 2	物件提議與追蹤	精確的物件邊界、動態追蹤
深度估計	Depth Anything	單張影像估深度	單目補足深度、稠密重建

表 32-2　五類 AI 視覺模型在 SLAM 中的角色

一個關鍵的工程限制

這裡有一個對機器人部署極為重要的現實：在穿戴裝置、機器人或智慧相機上的實用視訊系統，不可能為每一種能力各裝一個骨幹網路；既不能在能力上妥協，也無法承受記憶體與延遲的代價。

換句話說，雖然每個任務都有最佳的專用模型，但機器人的運算資源不允許同時跑五個大模型。實務上必須取捨——選一到兩個關鍵模型，或使用能兼顧多任務的統一骨幹。這正是第 25 章「硬體與演算法共同設計」思想在 AI 時代的延續。

對本書讀者的建議

- 入門：從 YOLO 開始。它成熟、輕量、資源豐富，能立即產生可見的成果（32.4 節的實作）。
- 進階：加入深度估計模型補足單目相機的深度（第 9.4 節），或用分割模型取得精確的物件邊界。
- 研究：探索開放詞彙模型，讓機器人能辨識訓練時未定義的物件——這是語意導航能否實用化的關鍵。

32.3　從 Visual SLAM 到 Semantic SLAM 到具身 AI 導航

本節梳理這條演進脈絡，讓同學看清自己所學的位置與前方的方向。

三個階段

5. 視覺 SLAM（第 20 至 23 章）：輸出相機軌跡與稀疏或稠密的幾何地圖。地圖中的每個點只有座標，沒有意義。
6. 語意 SLAM：在建圖的同時進行物件辨識與分割，把語意標籤附加到地圖元素上。地圖中的點不再只是「有東西」，而是「這是椅子的一部分」。
7. 具身 AI 導航：機器人不只有語意地圖，還能理解任務、推理常識、規劃行動。它能接受「去把會議室的燈關掉」這樣的指令並自主完成。

語意 SLAM 的兩種做法

- 後處理式：先用傳統 SLAM 建好幾何地圖，再把物件偵測的結果投影上去。實作簡單、模組解耦，是教學與專題的首選（32.4 節的實作路徑）。
- 緊耦合式：把語意資訊納入 SLAM 的估測過程——例如用物件層級的關聯取代點層級的關聯，或利用「牆是平面、地板是水平」等語意先驗來約束最佳化。精度更高但實作複雜。

圖 32-1　語意地圖的四個層次，以及一句話如何變成導航目標



語意地圖常被誤解成「在地圖上貼標籤」，但真正有用的語意是有階層的。圖的左半部把它分成四層：最底下是幾何層，只回答「這一格是空的還是被占住」；往上是物件層，把一叢點認定為一個實例，例如「這是滅火器」；再往上是區域層，把一群物件歸屬到一個有機能

的空間，例如「實驗室 B1」；最上層是建築層，記錄樓層與拓撲關係，例如「B1 在三樓，經 A 電梯可達」。

這個階層是由下而上建立的，順序不能顛倒：沒有正確的幾何，物件實例會落在錯的位置；沒有物件，區域就無從歸納。這也是為什麼本書把語意放在第九篇——它需要前面八篇全部到位才能成立。

圖的右半部則示範了階層的用途：由上而下解析一句話。使用者說「去三樓實驗室旁邊那個滅火器」，系統先在建築層選對樓層地圖，再到區域層找出「實驗室」，接著在該區域內的物件層搜尋「滅火器」，最後取出該實例的三維座標作為導航目標。每一層只需要回答一個簡單的問題，而不是讓一個模型一次處理整句話。

這正是三維場景圖的價值所在。若地圖只是一堆標了類別的格子，「實驗室旁邊」這個空間關係根本無法表達；而開放詞彙讓最後一步的搜尋不受固定類別清單限制——即使訓練時沒有「滅火器」這個類別，仍能以文字特徵比對找到它。

四個層次的實務意義

幾何層決定安全：撞不撞得到東西，只看這一層。語意出錯不致命，幾何出錯會。

物件層決定互動：能不能「去拿那個東西」，取決於實例是否被正確切分與定位。

區域層決定效率：有了房間概念，才能說「只掃廚房」或「先巡三樓」（第 16、19 章）。

建築層決定可達性：跨樓層的導航需要拓撲，而電梯是典型的拓撲節點而非幾何障礙。

專題建議：先做到物件層就很有價值，區域層可用第 16.4 節的距離變換法自動切分，建築層則往往用人工標注即可。

前沿：開放詞彙與場景圖

近年的研究正朝兩個方向推進：

- 開放詞彙線上語意建圖：讓 SLAM 系統不受預定義類別限制，能在建圖過程中辨識並標註任意物件——這對真實環境的複雜多樣至關重要。
- 場景圖表示：不只標註「這裡有一張椅子」，還記錄「椅子在桌子旁邊、桌子在會議室裡」的關係結構。近期研究已展示以多模態三維場景圖支撐零樣本的具身導航。

場景圖的價值在於它能支撐推理：當使用者說「去拿桌上的杯子」，機器人需要理解「桌上」這個空間關係——這在純幾何地圖中無法表達。

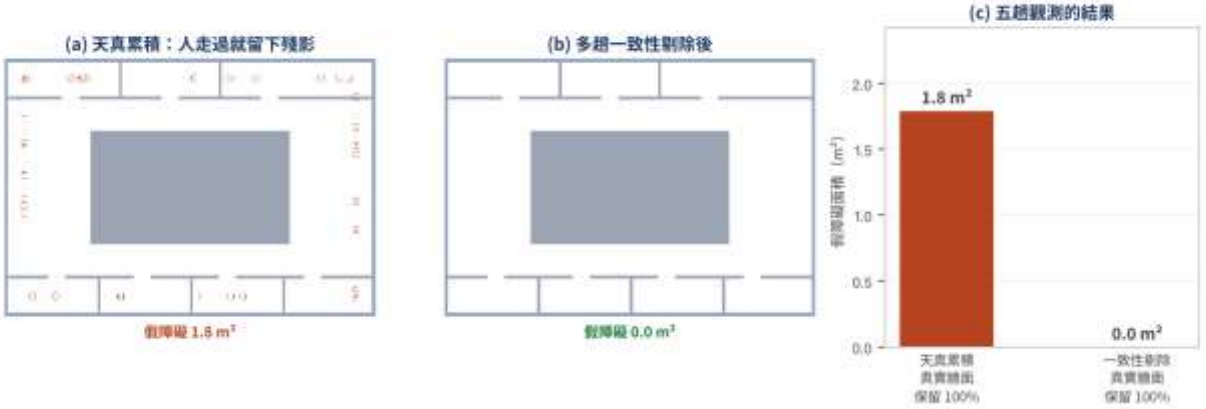
學術與產業的匯流

值得注意的是，這條演進線上，學術界與產業界正在快速匯流。相關研究已把 SLAM 與語意、三維高斯、鳥瞰視角、導航、大型語言模型、多階段作業等主題整合為同一個研究社群關注的範疇。對同學而言，這意味著：本書所學的幾何基礎，正是進入這個熱門領域的門票。

32.4　動態環境：會動的東西不該進地圖

語意最直接、也最早被產業採用的用途，其實不是「認出那是什麼」，而是「認出那個不該被記下來」。第 13.5 節談過佔據機率的上下限讓地圖保持可塑性，但那是被動的遺忘；有了語意，機器人可以主動判斷：這是人，所以不要寫進地圖。

圖 32-2　人走過留下的殘影，以及怎麼把它清掉



這個實驗讓機器人在同一層樓繞行五趟，每趟環境中都有七個隨機位置的人。圖(a) 是天真累積的結果：只要任何一趟觀測到某一格被占住，就把它記成障礙。灰色是真實牆面，紅色是地圖上多出來的假障礙——總共 1.8 平方公尺。它們散落在走廊各處，每一塊都是某個人在某一趟剛好站過的位置。

圖(b) 是加上「多趟一致性」判準之後：一格必須在五趟中至少三趟被觀測為占住，才會被記成障礙。結果是假障礙從 1.8 平方公尺降到 0，而真實牆面的保留率仍是 100%。道理很簡單——牆每一趟都在，人不會。

這個判準之所以有效，是因為它利用了一個語意上的事實：永久結構與臨時物體在時間上的行為完全不同。值得注意的是，這裡並沒有用到任何深度學習模型，只用了「多趟投票」這個統計方法——這正是第 19.6 節分層地圖策略的具體實作。

那麼什麼時候才真正需要 AI？答案是當你等不及五趟的時候。多趟投票需要時間，而機器人在第一趟就必須避開迎面走來的人。這時就需要即時的實例分割：在單一幀影像中認出「這是人」，直接把對應的點排除在建圖之外。兩種方法在實務上是並用的——AI 處理當下，投票處理長期。

動態物體的三道防線

即時層（毫秒）：以實例分割或物件偵測在單幀中剔除人、推車等已知動態類別。優

點是反應快，缺點是只認得訓練過的類別。

短期層（秒到分）：以連續幀之間的不一致性偵測移動物體，不需要類別知識，但對緩慢移動的物體不敏感。

長期層（趟與趟之間）：本圖的多趟一致性投票。不需要任何模型，能處理所有類型的變動，代價是需要時間累積。

三道防線解決的是不同時間尺度的問題，實務系統通常三者並用，而不是挑一個最好的。

32.5　語意如何改變導航決策

本節說明語意如何真的改變規劃器的行為，以及其中的陷阱。

從類別到成本：三種注入方式

- 區域層注入：把整個區域標上類別（禁區、人流熱區），再把類別映射成代價。最簡單，也最常用於人工設定的禁區。
- 物件層注入：偵測到特定物件（玻璃、鏡面、樓梯口）時，在其周圍加上一圈代價。適合處理感測器會失效的物體。
- 可通行性學習：直接從影像預測「這裡好不好走」，不經過明確的類別。近年在戶外與非結構化地形上相當流行，代價是難以解釋出錯的原因。

一個容易被忽略的陷阱：標註會貼歪

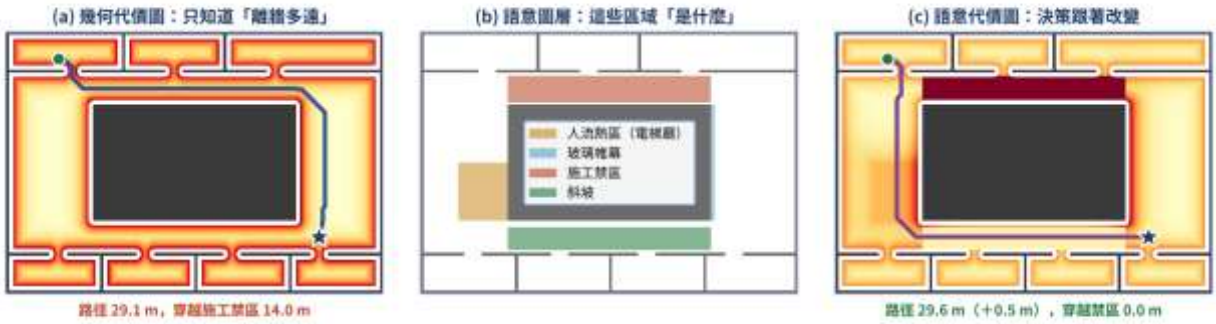
語意分割的準確率常被當成唯一指標，但對 SLAM 而言，「認得對」與「貼得準」是兩回事。一個把滅火器認得完美的模型，若相機與光達的外參差了兩度，標註落在地圖上的位置就會偏掉數十公分。

公式二已經寫出了這條傳播鏈：影像上的偵測結果要先經相機內參反投影、再經相機到底盤

的外參、底盤到里程計、里程計到地圖，才會落在最終的地圖座標上。這條鏈上任何一環的誤差都會累加。換句話說，語意地圖的品質同時取決於 AI 的辨識率與第 10 章的座標轉換品質——而後者往往才是實務上的瓶頸。

這帶出一個很實用的除錯順序：當語意標註貼歪時，先不要急著換更大的模型。先用一個已知位置的物件（例如貼在牆上的標靶）驗證外參與定位，確認幾何鏈沒問題之後，再去檢討辨識模型。本書第一至八篇的內容，在這裡仍然是地基。

圖 32-3　同一張地圖，加上語意之後決策就不一樣了



本書以一層回字形走廊的樓層做實驗：中央是電梯與機房構成的核心，四周環繞走廊，外圈是教室與實驗室。機器人要從左上角的房間送東西到右下角——繞核心左右兩側都走得通，於是「選哪一邊」就成了一個真正的決策。

圖(a) 是只有幾何資訊的代價地圖。顏色越紅代表越靠近障礙、成本越高，規劃器因此傾向走在走廊中央。它選了順時針的上方路線，全長 29.1 公尺——以純幾何的標準來看，這是正確答案。問題在於，這條路有 14.0 公尺穿過了施工禁區，而幾何地圖完全不知道那裡是禁區：對它而言，那只是一段沒有障礙物的走廊。

圖(b) 是加上語意圖層之後的同一層樓。四個區域被標示出來：電梯廳的人流熱區、玻璃帷幕、施工禁區，以及無障礙斜坡。這些資訊在幾何上完全看不出來——玻璃帷幕甚至是光達看不到的東西，它之所以出現在地圖上，正是因為有別的感測模態與語意模型把它標了出

來。

圖(c) 是把語意轉換成代價之後的結果。禁區被設成不可通行、人流熱區與玻璃帷幕被加上隨距離衰減的額外成本、斜坡則因為好走而降低成本。規劃器於是改走逆時針的下方路線，全長 29.6 公尺——只多了 0.5 公尺，卻把 14 公尺的禁區穿越降為零。這 0.5 公尺就是語意的價格，而它買到的是一條合規、安全、也更符合人類預期的路線。

語意代價怎麼設才合理

硬約束 vs 軟成本：禁區、施工區應設為不可通行（硬約束）；人流熱區、玻璃帷幕則適合設為可被權衡的額外成本（軟成本）。

衰減距離比數值更重要：本圖對玻璃帷幕設 1.0 公尺的衰減距離、人流熱區 1.3 公尺。這個距離決定了機器人「離多遠才開始閃避」，比成本數值本身更影響行為。

負成本也有用：無障礙斜坡被設為低成本，等於主動鼓勵機器人走它。語意不只用來避開壞的地方，也用來偏好好的地方。

小心過度加權：若把每個語意區都設成高成本，規劃器會變得保守甚至找不到路。實務上建議一次只加入一到兩層語意，確認行為合理後再疊加。

32.6　國際研究地圖：語意 SLAM 的三條路線

語意 SLAM 在國際上大致沿三條路線發展，各有代表性的研究群與系統。本節提供一張地圖，讓同學知道該往哪裡找資料。

一、度量—語意 SLAM：把語意接進既有的 SLAM 框架

這條路線的主張是：不要另起爐灶，而是在成熟的 SLAM 系統上加一層語意。代表性的工作把實例分割接到視覺慣性 SLAM 的後端，在建立幾何地圖的同時維護物件層的實例，並以三維場景圖組織它們。優點是繼承了既有系統的穩定度，也最符合本書第一至八篇的架構。

二、開放詞彙地圖：擺脫固定類別清單

傳統語意分割只能認出訓練時定義的那幾十個類別，但真實場域裡的東西遠不止於此。開放詞彙這條路線把視覺語言模型的特徵直接存進地圖的每一個元素，查詢時再以文字比對——於是「找紅色的滅火器」這種事先沒定義過的查詢也能運作。代價是記憶體用量大幅上升，而且查詢結果難以保證。

三、神經場與可算繪表示：定位與重建合一

第 17.8 與第 20.8 節都提過的三維高斯潑濺，在語意領域也有對應發展：把語意特徵一併存進每個高斯元素，讓地圖既能算繪出逼真影像、又能回答語意查詢。這條路線在重建品質上最有優勢，但運算與記憶體成本最高，目前多用於離線重建而非即時導航。

三條路線的取捨可以這樣理解：第一條最穩、最適合部署；第二條最靈活、最適合開放場域；第三條最漂亮、最適合展示與離線分析。對專題而言，建議從第一條開始——它與本書的技術棧完全相容，而且出錯時你知道該去看哪一層。

怎麼追這個領域的進展

會議：ICRA、IROS、RSS、CVPR、CoRL 是語意 SLAM 的主要發表場域。

關鍵字：semantic SLAM、open-vocabulary mapping、3D scene graph、instance-aware mapping、dynamic SLAM。

評估資料集：室內以 ScanNet、Replica 為主；戶外與自駕則看 SemanticKITTI、nuScenes。

實作起點：先在既有的 ORB-SLAM 或 RTAB-Map 上接一個現成的分割模型，把結果投影到地圖，親手走一遍公式二的傳播鏈——這比讀十篇論文更有感覺。

公式或流程

一、物件偵測的評估指標

訓練 YOLO 等偵測模型時，最常用的指標是平均精度均值：

$$mAP = (1/C) \Sigma_c AP_c$$

其中 C 為類別數、AP_c 為第 c 類的平均精度（精確率對召回率曲線下的面積）。常見的變體是以交併比門檻 0.5 計算的 mAP@0.5，以及在多個門檻上平均的 mAP@0.5:0.95（較嚴格）。

對機器人應用而言，還應關注兩個實務指標：

- 漏檢率：沒偵測到的比例。對安全相關的物件（人、樓梯口），漏檢比誤檢危險得多。
- 每類別的表現：總體 mAP 高不代表每一類都好。若「樓梯」這一類的精度特別低，而它正是安全關鍵，那麼整體數字就有誤導性。

二、把偵測結果投影到三維地圖

語意標註的核心運算，是把影像上的偵測框轉為三維座標。以偵測框中心像素（u, v）與其深度 d 為例，依第 30 章公式一反投影：

$$P_{camera} = ((u - cx) \cdot Z / fx,\ (v - cy) \cdot Z / fy,\ Z) \quad ,\ Z = d / scale$$

再依第 10 章的座標轉換鏈，轉到地圖座標系：

$$P_{map} = T_{map \leftarrow odom} \cdot T_{odom \leftarrow base} \cdot T_{base \leftarrow camera} \cdot P_{camera}$$

這條式子把本章的 AI 與第 10 章的座標轉換連在一起——語意標註看似是 AI 的工作，但它能否正確落在地圖上，取決於外參與定位的品質。這再次印證 32.1 節的觀點：語意層必須建立在穩固的幾何層之上。

四、AI 輔助 SLAM 的完整資料流

圖 32-4　從感測到語意決策的端到端流程

① 感測器：相機、光達、IMU

↓

② 幾何 SLAM：定位與建圖（第 13–22 章）→ 位姿 + 幾何地圖

↓

③ AI 感知：YOLO 偵測 / 分割 / 深度估計（32.2 節）

↓ 依公式二把偵測結果投影到地圖座標

④ 語意地圖：幾何 + 物件標籤 + 空間關係

↓

⑤ RAG 知識庫：校園拓樸、地點屬性、通行規則（32.5 節）

↓

⑥ VLM / LLM Agent：理解指令、推理、選擇目標（32.6 節）

↓ 輸出結構化的幾何目標

⑦ 路徑規劃與控制（第 29 章）→ 速度指令

↓

⑧ 執行 → 環境改變 → 回到 ①

【安全把關】步驟 ⑦ 有權否決 ⑥ 的決策

案例

案例

案例 32-1　語意對了，位置錯了

一個在專題中極常見的失敗：同學接上了準確率九成以上的物件偵測模型，機器人也確實認出了走廊上的滅火器，但在地圖上一看，標註點飄在牆外一公尺處，甚至落到隔壁房間去了。

直覺反應通常是「模型不夠好」，於是換更大的模型、重新標更多資料。但真正的原因幾乎都不在模型。偵測給的是影像上的一個框，要把它變成地圖上的一個點，必須經過深度估計、相機內參、相機到底盤的外參、以及當下的定位結果——四個環節。其中最常出錯的是外參：相機裝上去之後沒有重新校正，或是用了設計值而非實測值，差個兩三度就足以造成數十公分的偏移。

診斷方法很簡單：把一個已知位置的物件（貼在牆上的標靶）當成測試對象，讓機器人從不同角度偵測它十次，看標註點是否收斂在同一個位置。若標註點隨視角散開成一片，問題在外參或定位；若集中但整體偏移，問題在外參的平移；只有在標註點本身就跳來跳去時，才需要回頭檢討模型。

這個案例的教訓與第 19.5 節的重複定位精度相同：AI 的輸出要落在正確的地方，靠的是第 10 章的座標轉換與第 11 章的定位。語意層再強，也站在幾何層的肩膀上。

案例 32-2　為什麼掃地機器人不把你的腳記成牆

一個每天都在發生、卻很少被注意的語意應用。掃地機器人工作時，家裡的人會走來走去；若它把每一次偵測到的腳都記成障礙，地圖很快就會布滿不存在的牆，第 16 章談的覆蓋規劃也會跟著失效。

它的做法正是圖 32-2 的三道防線。即時層：以紅外線或相機判斷前方是移動物體，暫時繞開但不寫入地圖。短期層：同一位置在連續幾次掃描中時有時無，判定為不穩定觀測。長期層：多次清掃都沒再出現的障礙，其佔據機率會被逐步降低，最終從地圖上消失（第 13.5 節的上下限機制）。

值得注意的是，低階機型往往只有長期層，所以你會看到它把臨時放在地上的背包記成障礙，而且好幾天都繞著那個已經不存在的東西走。這個差異就是語意能力在產品上的具體價值——

—它不一定表現為「更聰明」，而是表現為「比較不會做蠢事」。

案例 32-3　開放詞彙的代價

開放詞彙地圖聽起來近乎萬能：不必事先定義類別，使用者想找什麼就用文字描述。但它的代價相當具體，值得同學事先知道。

其一是記憶體。傳統語意地圖每個元素只存一個類別編號（幾個位元組），開放詞彙地圖則要存一個視覺語言模型的特徵向量——常見維度是數百維的浮點數，也就是每個元素上千位元組。同樣一張地圖，記憶體用量可能差上兩三個數量級，這正是第 33 章要處理的邊緣部署難題。

其二是可靠性。固定類別的模型至少有明確的評估指標與失敗邊界；開放詞彙查詢則可能對相近的描述給出完全不同的結果，而且沒有一個好方法可以事先知道哪些查詢會失敗。對需要安全保證的導航任務而言，這是個嚴重的問題。

目前的務實做法是混合：安全關鍵的類別（人、樓梯、禁區）仍用固定類別的模型、有明確的評估與保證；開放詞彙則用於使用者互動與查詢這類「錯了也不會出事」的場合。這個分工原則同學可以直接套用在專題上。

實作活動

實作活動

實作 32-1　語意代價地圖與動態物體剔除

目的：親手把語意資訊接進地圖與規劃器，驗證它如何改變決策，並實作動態物體的多趟一致性剔除。本實作以 Python 為主，第一、二部分不需要機器人硬體。

第一部分：語意代價地圖（全體）

1. 取一張佔據格地圖（可用第 13 章實作的成果），以距離變換產生幾何代價圖，並用

A* 規劃一條起點到終點的路徑，記錄長度。

2. 在地圖上以矩形標出二至三個語意區域（禁區、人流熱區、偏好路線），為每一類指定成本與衰減距離。

3. 把語意代價疊加到幾何代價圖上，重新規劃，比較兩條路徑的長度、以及各自穿越語意區域的距離，重現圖 32-3 的三個數字。

4. 敏感度實驗：把人流熱區的成本從 0 逐步加大，找出路徑「跳」到另一條路線的臨界值，並解釋這個臨界值的意義。

第二部分：動態物體剔除（全體）

1. 在模擬環境中隨機放置數個「人」，執行一趟掃描建圖，重複五趟。

2. 以天真累積（任一趟觀測到即記為障礙）建圖，與真實地圖比對，計算假障礙的面積，重現圖 32-2(a)。

3. 改用多趟一致性投票（至少 k 趟觀測到才記為障礙），掃描 k 從 1 到 5，畫出「假障礙面積」與「真實牆面保留率」兩條曲線，找出最佳的 k。

4. 討論：若把趟數從 5 增加到 20，最佳的 k 會怎麼變？這對巡檢機器人（第 19 章）的地圖維護策略有什麼啟示？

第三部分：語意標註投影（大學部）

1. 以 RGB-D 相機或模擬資料，對一張影像執行現成的物件偵測模型，取得偵測框與深度。

2. 依公式二把偵測結果投影到地圖座標系，把標註點畫在地圖上。

3. 刻意把外參的旋轉改動 1、2、5 度，記錄標註點偏移了多少，量化外參誤差的影響，重現案例 32-1 的現象。

4. （進階）對同一個物件從五個不同視角偵測並投影，計算標註點的離散程度，作為整條傳播鏈品質的綜合指標。

討論問題

1. 語意路徑比幾何路徑多走了多少？這個代價換到了什麼？若多走的距離是 50% 而不是 2%，你還會選語意路徑嗎？
2. 多趟一致性投票的最佳 k 值是多少？k 太小與太大各會發生什麼？
3. 外參旋轉差 2 度時，十公尺外的標註點偏移多少？這個數字與第 19.5 節要求的公分級定位精度相比如何？
4. 哪些類別適合用固定清單的模型、哪些適合開放詞彙？請以「出錯的後果」為判準分類。
5. （大學部）若要為校園導覽機器人設計語意地圖，你會放入哪四個語意層？每一層的成本與衰減距離怎麼設？

延伸挑戰（大學部）

其一：實作一個「語意圖層編輯器」——讓使用者在地圖上畫出區域、選擇類別、調整成本，並即時看到規劃路徑如何改變。這類工具是巡檢與服務機器人產品的標準配備。其二：把動態物體剔除從「投票」升級為「分層地圖」：維護永久層與暫態層兩張地圖，定位只用永久層、避障兩層都用，並比較它與單純投票法在定位成功率上的差異（可沿用第 19.6 節的評估方法）。

延伸閱讀

1. N. Yokoyama et al., "VLFM: Vision-Language Frontier Maps for Zero-Shot Semantic Navigation," ICRA, 2024.（零樣本語意導航的代表作，第 24、32 章共同參考）
2. T. B. Martins, M. R. Oswald, and J. Civera, "Open-Vocabulary Online Semantic Mapping for SLAM," IEEE RA-L, 2025.（開放詞彙語意建圖，32.3 節的前沿方向）

3. D. Shah et al., "LM-Nav: Robotic Navigation with Large Pre-trained Models of Language, Vision, and Action," CoRL, 2022.（語言模型驅動導航的開創性工作）
4. Embodied AI with Foundation Models for Mobile Service Robots: A Systematic Review, arXiv, 2025.（具身智慧與基礎模型的完整綜述，適合作為研究起點）
5. Ultralytics YOLO 官方文件（docs.ultralytics.com）.（物件偵測模型的訓練、評估與部署，本章實作的主要工具）
6. awesome-and-novel-works-in-slam 開源清單（GitHub）.（語意 SLAM、三維高斯、導航、大型語言模型等新方向的持續更新整理）
7. NVIDIA Jetson 開發者文件與 TensorRT 最佳化指南.（邊緣 AI 部署的實務參考，第 33.8 節的技術基礎）

小結

小結

- 傳統 SLAM 知道形狀卻不知道意義；機器人智慧可分幾何、語意、任務三個層次，三者是疊加而非取代——語意層若無穩固的幾何層支撐，標籤只會貼歪。
- AI 視覺模型在 SLAM 中有五類分工：偵測、分割、深度、描述與場景理解；選型的關鍵限制是邊緣平台的運算與記憶體預算。
- 語意地圖可分四個層次：幾何層、物件層、區域層與建築層，由下而上建立；開放詞彙讓類別不必事先定義，場景圖則讓空間關係可被查詢與推理。
- 動態物體若直接寫入地圖會留下殘影。本章實驗顯示，天真累積產生 1.8 平方公尺的假障礙，而多趟一致性投票可將其降為零，同時保留 100% 的真實牆面。
- 動態物體的處理有三道防線——即時的實例分割、短期的幀間不一致、長期的多趟投票，三者處理不同時間尺度，實務上並用而非擇一。
- 語意進入導航是透過代價地圖。本章實驗顯示，語意路徑只多走 0.5 公尺（29.1 →

29.6 公尺），卻把 14 公尺的禁區穿越降為零；硬約束與軟成本應分開設計，而衰減距離比成本數值更影響行為。

- 語意標註的品質同時取決於辨識率與座標轉換鏈；外參差兩三度就足以造成數十公分的偏移，除錯時應先驗證幾何鏈再檢討模型。
- 國際研究沿三條路線發展：度量—語意 SLAM 最穩定、開放詞彙最靈活、神經場與可算繪表示重建品質最佳但成本最高。

承先啟後

本章把地圖從「形狀」推進到「意義」。圖 32-1 整理了語意地圖的四個層次、圖 32-2 說明了哪些東西不該進地圖、圖 32-3 則示範了語意如何改變決策。但所有這些都預設了一件事：你手上已經有一個夠好的 AI 模型。

模型從哪裡來？真實資料昂貴、標註耗時，而機器人在真實世界裡試錯的代價更高。下一章我們進入模擬：第 33 章介紹如何在模擬環境中大規模訓練機器人，再把學到的策略部署到真實的邊緣硬體上——這條「先模擬、再部署」的路線，已經是從國家級專案到產業量產的標準流程。

（本章到此結束，下一章進入模擬訓練與邊緣部署。）

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 33 章

模擬訓練與 Sim-to-Real

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程　/　高中 AI 機器人社團課程

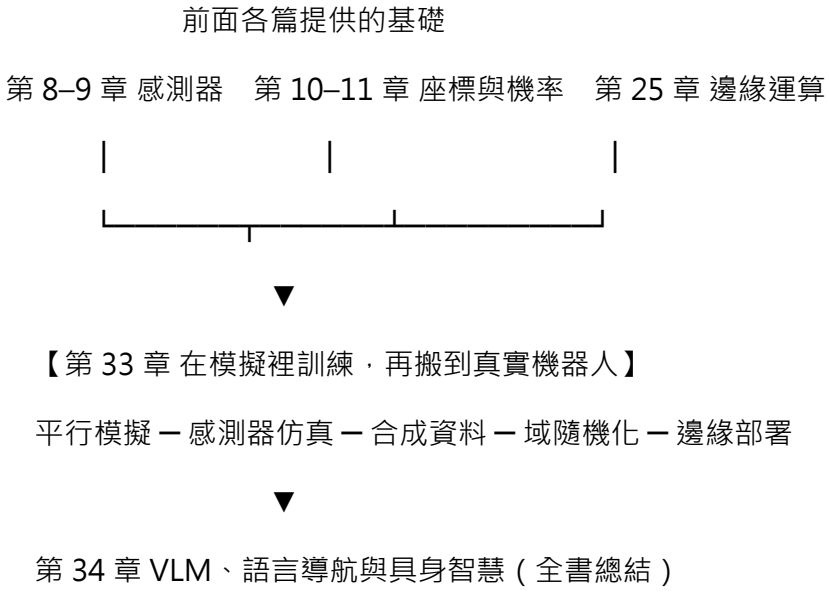
本章在全書的位置

第 32 章讓地圖有了意義——機器人知道「那是一扇門」。但知道之後呢？要讓機器人真的走過去、把門推開、在不平的地面上站穩，背後需要一套控制策略，而這套策略幾乎不可能在真實機器人上從零學會。原因很簡單：強化學習需要幾千萬次的嘗試，真實機器人摔不起那麼多次，也沒有那麼多時間。

本章要處理的就是這個矛盾：先在模擬裡練，再把學到的東西搬到真實機器人上。這條路線稱為 sim-to-real，它在 2020 年之後幾乎成為足式機器人與靈巧操作的標準做法。本章以輝達的 Isaac Sim 與 Isaac Lab 生態為主線，因為它是目前最完整、也最容易取得的一套工具；但本章要講的觀念——平行模擬為什麼有效、模擬與真實的落差從哪裡來、怎麼把落差補起來——不隨工具改變。

本章與前面各篇的關係很直接：第十、十一章的座標與機率是模擬器的語言，第八、九章的感測器模型決定模擬器要仿真什麼，第 25 章的邊緣運算則是本章 33.8 節「訓練完怎麼部署」的硬體基礎。可以說，本章是把全書的技術從「能跑」推到「能學」的一章。

圖 33-0　第 33 章的連結地圖



學習目標

本章從「為什麼要先模擬」出發，依序處理平行模擬的原理、感測器仿真、合成資料生成、sim-to-real 落差的來源與對策，最後落到邊緣裝置的部署。本章有三張以實驗算出來的圖，分別回答三個問題：平行度到底帶來什麼、模擬跟真實差在哪、以及域隨機化付出了什麼代價。讀完本章，同學應能：

1. 說明為什麼強化學習的機器人策略幾乎都必須先在模擬中訓練，並能估算所需的互動次數。
2. 說明 Isaac Sim 與 Isaac Lab 的分工，以及 GPU 原生模擬與傳統模擬器的差異。
3. 解釋平行環境數如何同時影響吞吐量、梯度雜訊與訓練穩定度，並知道三者各自的極限。
4. 說明機器人常用感測器在模擬中如何被仿真，以及哪些部分仿真得好、哪些仍有落差。
5. 說明合成資料生成與真實場景重建的做法，並能為自己的專題規劃資料來源。
6. 指出 sim-to-real 落差的主要來源，並能分辨「系統識別」與「域隨機化」兩種對策的適用時機。
7. 說明域隨機化的代價：穩健度與名目效能之間的取捨。
8. 規劃一條從雲端訓練到 Jetson 部署的完整路徑，並能估算量化後的記憶體需求。

核心概念

名詞	意義	關聯章節
Sim-to-Real	在模擬中訓練策略，再部署到真實機器人	33.1、33.6
平行模擬	在同一張 GPU 上同時推進數千個環境	33.3

GPU 原生	狀態以張量形式常駐 GPU，不經 CPU 往返	33.3
OpenUSD	描述場景與機器人的開放場景格式	33.2
域隨機化	訓練時隨機抽樣環境參數，換取部署穩健度	33.6
系統識別	量測真實機器人並反推模擬參數	33.6、第 11 章
合成資料	由模擬器產生並自動標註的訓練資料	33.5、第 32 章
域落差	訓練分布與部署分布不一致造成的效能下降	33.5、33.6
量化	以較少位元表示模型參數以節省記憶體	33.8
蒸餾	把大模型的能力轉移到可在邊緣執行的小模型	33.8

表 33-1　本章的核心名詞

33.1　為什麼機器人要先在模擬裡學

先把問題的規模講清楚。

一、數字的問題

以強化學習訓練一個足式機器人的行走策略，典型的樣本需求是十億（10^9）等級的環境互動步數。假設控制頻率是 50 Hz，十億步相當於 2,000 萬秒，也就是約 231 天的連續運行。這還是在機器人不會壞、電池不用充、摔倒後會自己站起來的理想假設下。

真實世界還有三個模擬沒有的成本：硬體會壞（一次摔倒可能損壞減速機或外殼）、人力要

全程看顧（不能無人運行）、以及實驗不可重複（今天的地板濕度跟昨天不同，無法乾淨地比較兩個演算法）。這三點加起來，使得「在真機上做強化學習」在絕大多數情況下不是成本問題，而是根本不可行。

二、模擬提供了什麼

- 時間可以加速：模擬不受真實時間限制，一秒鐘可以推進數千個環境的數十步。
- 失敗沒有成本：摔倒只是把狀態重設，不會壞零件。
- 狀態完全可知：模擬器知道機器人的真實位姿與速度，這讓「非對稱訓練」成為可能——評價網路可以看到真實狀態，而策略網路只能看到感測器資料。
- 實驗可以重複：固定隨機種子，同一個實驗可以跑一百次，這是比較演算法的前提（第 31 章的評估方法）。
- 環境可以任意生成：地形、光線、物件擺放都能程式化地產生，這是真實世界做不到的多樣性。

三、模擬不提供什麼

必須同樣誠實地講清楚模擬的限制。模擬器解的是你寫進去的方程式，不是真實世界的物理。凡是沒有寫進方程式的東西——馬達的溫升、齒輪的背隙、線材的拉扯、腳墊磨損後摩擦係數的改變、韌體裡那些工程師為了好用而加上的補償——模擬器都不知道。這些「未建模的動力學」正是 sim-to-real 落差的主要來源，33.6 節會逐一處理。

> **為什麼 SLAM 比控制更晚用上模擬訓練**
>
> 本書前三十二章的 SLAM 演算法，大多不需要訓練——它們是推導出來的，不是學出來的。這是 SLAM 與控制策略的根本差異：SLAM 有明確的機率模型與最佳化目標（第 11、12 章），而「怎麼在碎石地上走穩」沒有人寫得出閉式解。
>
> 但這條界線正在模糊。深度學習的視覺里程計、學習式的迴路閉合描述子、以及第 32 章的語意分割，都需要大量資料；而 33.5 節的合成資料生成，正是讓 SLAM 也

開始受益於模擬的途徑。

33.2　Isaac Sim 與 Isaac Lab 是什麼

這兩個名字經常被混用，但分工其實很清楚。

一、兩層的分工

	Isaac Sim	Isaac Lab
定位	模擬器與場景編輯環境	機器人學習的框架
負責	物理求解、感測器算繪、場景與機器人匯入	任務定義、觀測與獎勵、訓練流程
使用者	要做場景、要接 ROS 2、要產生資料的人	要訓練策略的人
底層	PhysX 物理 + RTX 算繪 + OpenUSD 場景	建構在 Isaac Sim 之上
類比	實驗室與儀器	實驗手冊與評分表

表 33-2　Isaac Sim 與 Isaac Lab 的分工

講得更白一點：Isaac Sim 是「世界」，Isaac Lab 是「課程」。你可以只用 Isaac Sim 做一個場景、放一台機器人、接上 ROS 2 來測試你的導航堆疊，完全不碰強化學習；也可以用 Isaac Lab 直接取用它內建的數十個訓練環境，不必自己從零搭場景。

二、三根支柱

- PhysX：物理求解器，處理剛體、關節、接觸與摩擦。對機器人而言最關鍵的是接觸求解的品質——腳掌踩地、夾爪抓物都靠它。
- RTX：光線追蹤算繪，負責產生相機影像。它同時也是光達與深度相機仿真的基礎（33.4 節）。

- OpenUSD：開放的場景描述格式，由皮克斯提出、現已成為跨工具的場景交換標準。它的價值在於「一份場景、多個工具共用」，也讓機器人模型（連桿、關節、致動器、感測器）能以標準結構描述。

三、2025 年的重要變化

2025 年 8 月 11 日的 SIGGRAPH，Isaac Sim 5.0 與 Isaac Lab 2.2 開放一般取用，同時把 Isaac Sim 的擴充套件開源到 GitHub（isaac-sim/IsaacSim 與 isaac-sim/IsaacLab）。這對教學與研究是一件大事：原本只能當成黑盒子使用的模擬器，現在可以讀原始碼、可以改、也可以提交修改。

但有一點要說清楚，免得同學踩空：底層的 Omniverse Kit 並未全部開源，目前也不接受外部提交。也就是說，你可以修改 Isaac Sim 的擴充層，但不能修改最底層的核心。對課程專題而言這通常不構成限制，但在規劃長期研究時值得先知道。

同一版本也帶進了幾項與機器人直接相關的功能：OpenUSD 的機器人結構描述與匯入精靈，讓 URDF 轉換更不容易出錯；關節摩擦模型，讓減速機的真實阻力可以被寫進模擬；以及 ROS 2 Jazzy 的支援與標準化的模擬介面，讓同一套 ROS 2 節點可以在不同模擬器之間切換（第八篇）。

Isaac Lab 2.2 帶進來的東西

可一鍵啟動的雲端訓練環境（Brev launchable），不必自己裝 CUDA 與驅動程式。

GR00T-Mimic 的雙臂訓練環境，示範從人類示範資料學習操作。

Omniverse Fabric 的整合，降低大規模場景的記憶體與同步開銷。

張量化的吸盤夾爪模型，讓真空吸取的任務能平行模擬。

開源的策略評估框架，讓訓練完的策略有一套標準的比較方式（呼應第 31 章）。

33.3　GPU 原生的平行模擬

這是整個生態的技術核心，也是本章第一張實驗圖要回答的問題。

一、「GPU 原生」的意思

傳統模擬器（如 Gazebo）的流程是：CPU 算物理 → 把狀態複製給學習程式 → 學習程式算出動作 → 再複製回模擬器。每一步都要在 CPU 與 GPU 之間搬資料，當環境數量一多，搬資料的時間就會超過算物理的時間。

GPU 原生的做法是：把所有環境的狀態以張量形式直接存在 GPU 記憶體裡，物理求解、觀測計算、獎勵計算、神經網路推論全部在 GPU 上完成，整個訓練迴圈不需要把資料搬回 CPU。這個差別在環境數少的時候看不出來，但在數千個環境時就是數量級的差距。

二、平行度到底帶來什麼

「多開幾個環境就會比較快」這句話人人會說，但快在哪裡、快多少、以及什麼時候就不再變快，是三個不同的問題。以下用一個可以完整重現的實驗把三件事分開來看。

實驗設定：一個單自由度的關節追蹤任務。關節有轉動慣量、黏滯阻尼、庫倫摩擦與致動延遲四個參數；控制器是一組含摩擦前饋的 PD 增益，也就是要「學」的三個參數。訓練方式是最單純的隨機爬山法，每次迭代用 N 個隨機抽樣的環境來評估候選參數。這個任務比真正的足式運動簡單得多，但平行度的三個效果在這裡已經完全呈現。

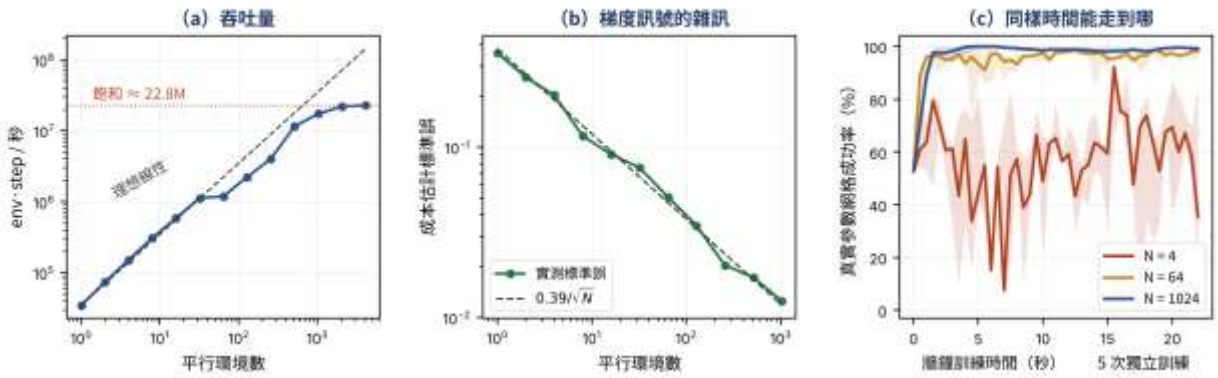


圖 33-1　平行環境數的三個效果：吞吐量、梯度雜訊與訓練穩定度

先看（a）。橫軸是同時推進的環境數 N，縱軸是每秒能推進的環境步數，兩軸都是對數刻度。虛線是「理想線性」——如果平行度完全免費，加倍環境數就該加倍吞吐量。實測結

果在 N 小於約 32 時幾乎貼著理想線走：從 N=1 的每秒 3.5 萬步，一路漲到 N=32 的每秒 113 萬步，三十二倍的環境換來三十二倍的吞吐量。這段之所以「免費」，是因為單一環境根本填不滿 GPU，多開的環境只是把原本閒置的運算單元用起來。

N 超過 32 之後曲線開始偏離虛線，到 N=4096 時吞吐量停在每秒約 2,285 萬步。與 N=1 相比是 654 倍，但與理想線性的 1.43 億步相比只有六分之一。飽和的原因是 GPU 的運算單元已經被填滿，再加環境只能排隊。值得注意的是：從 N=1 到 N=4096，每一批次的牆鐘時間只從 0.023 秒漲到 0.143 秒，也就是說環境數增加 4,096 倍，等待時間只增加 6.3 倍。這正是平行模擬最直觀的價值——幾乎白拿的樣本。

（b）回答第二個問題：這些樣本拿來做什麼。強化學習每次更新都要估計「這組參數有多好」，而這個估計是從有限個環境的平均算出來的，必然帶有雜訊。縱軸是這個估計的標準誤，實測點幾乎完美落在 1/√N 的虛線上——把每個點的標準誤乘上 √N，十一個點的結果都落在 0.32 至 0.43 之間。從 N=1 到 N=1024，標準誤由 0.357 降到 0.0126，正好接近 √1024 = 32 倍。這就是公式一。

（c）把前兩件事合起來看：給定相同的牆鐘時間預算（22 秒），不同平行度能走到哪裡。為了避免單次訓練的運氣成分，每個設定跑五次獨立訓練，圖中粗線是中位數、淡色帶是四分位距。N=4 的中位數只到 35.3%，而且四分位距從 15.8% 橫跨到 85.6%——換句話說，你不知道這次訓練會成功還是失敗。N=64 的中位數是 98.4%，N=1024 是 99.2%，且四分位距只有 98.3% 到 99.4%。

（c）要傳達的重點不是「平行度高比較快」，而是「平行度高比較可靠」。N=4 不是走得慢，是走得亂——梯度雜訊太大，每一步都可能走錯方向，最後落在哪裡幾乎是運氣。這個現象在真實的強化學習訓練中更嚴重，也是為什麼 Isaac Lab 這類框架把「能同時開幾千個環境」當作核心指標。

一個誠實的但書

上面三張圖是在一個單自由度的玩具任務上算出來的，不是足式機器人。真實任務的絕對數字會完全不同：飽和點取決於你的 GPU 與環境的複雜度，所需的環境數取決於動作維度與獎勵的稀疏程度。

但三個定性結論是通用的：吞吐量先線性成長後飽和、梯度雜訊依 1/√N 下降、以及平行度不足時訓練結果的變異會大到無法比較。同學在自己的任務上做同樣的三張圖，是規劃運算資源最有效的方法。

三、真實規模的數字

作為對照，看一下工業級的數字。Isaac Lab 的技術報告（2025 年 11 月）在 8 張 GPU、每張 16,384 個環境的配置下，靈巧手抓取任務的教師策略達到每秒超過 90 萬步，Franka 機械臂開櫃任務則超過每秒 160 萬步，並且在多機多卡的情況下接近線性擴展。

另一個對同學更實際的數字是：在開櫃任務上，一張消費級的 GeForce 5090 的吞吐量約等於兩張專業卡；在靈巧手任務上甚至快約 25%，因為該任務的瓶頸在 CPU 而非 GPU。這意味著一台配備單張高階顯示卡的桌機，已經足以支撐相當認真的機器人學習研究——這在五年前是不可想像的。

四、公式一：平行度與估計品質

設單一環境的回報估計為隨機變數，其標準差為 σ；以 N 個獨立環境的平均作為估計時，估計量的標準誤為：

$$\mathrm{SE}(N) = \sigma \,/\, \sqrt{N}$$

要把估計的雜訊降為原來的十分之一，需要一百倍的環境數。這個「平方根的懲罰」解釋了為什麼平行度要以千為單位計算：從 64 個環境提升到 4,096 個環境，雜訊只降為八分之一。它同時也解釋了（a）圖的飽和為什麼可以接受——即使吞吐量只剩理想值的六分之一，多出來的環境仍在降低雜訊。

33.4　感測器模擬：從 RTX 光達到觸覺

對本書的讀者而言，這一節大概是最切身的：前面三十二章用到的每一種感測器，在模擬裡長什麼樣子？

一、感測器的四個家族

家族	模擬中的實作	對應章節	仿真品質
本體感測	直接由物理狀態讀出	第 7、10 章	高
相機	RTX 光線追蹤算繪	第 9、20–23 章	中高
測距	Warp 光線投射	第 8、17–18 章	中
接觸與觸覺	接觸求解器 + 視觸覺模型	第 19 章	中低

表 33-3　感測器在模擬中的四個家族

二、逐一來看

本體感測最容易，也仿真得最好。IMU、關節編碼器、接觸偵測、座標轉換（frame transformer）都可以直接從物理狀態讀出來，再依第 7 章的誤差模型加上偏差與雜訊即可。但要注意：模擬器給的是「乾淨的真值加上你寫的雜訊」，而真實 IMU 的誤差有溫度漂移與振動耦合，不是獨立同分布的高斯雜訊。

相機是 RTX 算繪的主場。除了 RGB，還能免費拿到深度、語意分割、實例分割、表面法向量與運動向量——這些在真實世界需要額外標註的資料，在模擬裡是算繪管線的副產品。這正是 33.5 節合成資料的技術基礎。鏡頭模型同時支援針孔與魚眼，後者對第 9 章提過的廣角視覺導航很重要。

測距感測器（光達、深度相機的立體匹配）用的是 Warp 光線投射：從感測器原點依指定的掃描樣式射出光線，算出與場景的交點。這能正確處理遮擋與幾何，也能仿真各家光達不同

的線束排列；足式機器人常用的「地形高度掃描」也是用同一套機制實作。但有兩件事它不容易做對：第 8 章談過的鏡面反射與穿透（玻璃、水面、高反射標牌），以及第 18 章談過的運動畸變——後者需要模擬器在掃描期間持續更新感測器位姿，而不是在一個瞬間射出所有光線。

接觸與觸覺是目前落差最大的一類。接觸力本身由 PhysX 求解，但真實的接觸牽涉到材料的微觀變形、表面粗糙度與黏滯效應，這些在剛體求解器裡只能用摩擦係數近似。視觸覺感測器（以相機拍攝彈性膜的變形來推算接觸力）的模擬已經可用，但如果同學的專題要做精細的插拔或裝配，必須預期這一類任務的 sim-to-real 落差最大。

> **一個常被忽略的細節：感測器的時間**
>
> 真實感測器不是同時觸發的。光達轉一圈要 100 毫秒、相機曝光要幾毫秒、IMU 是 200 Hz、輪速編碼器是 50 Hz——這些在第 7 章與第 18 章都討論過。
>
> 模擬器預設常把所有感測器當成同一瞬間取樣。如果你的演算法對時間同步敏感（例如緊耦合的光達慣性里程計），那麼「模擬裡好好的、真機上就飄」很可能不是演算法的問題，而是模擬沒有把時間差建進去。
>
> 對策是在模擬中明確設定各感測器的更新率與相位，並刻意加入隨機的時間抖動。

33.5　合成資料生成與真實場景重建

第 32 章談過自己蒐集資料的必要性，也談過標註的成本。這一節提供另一條路：讓模擬器把資料跟標註一起生出來。

一、合成資料的誘惑與陷阱

誘惑很明顯：在模擬裡，每一個像素屬於哪個物件是已知的，因此分割標註、邊界框、深度圖、六自由度位姿全部免費且零誤差。想要一萬張不同光線、不同視角、不同物件擺放的訓練影像，只是一段腳本與幾個小時的運算。

陷阱也同樣明顯：模擬出來的影像和真實影像不一樣。材質的反射、感測器的雜訊與壓縮痕跡、鏡頭的髒污與色偏、真實場景裡那些沒有人會刻意建模的雜物——這些差異合稱域落差（domain gap）。一個只在合成資料上訓練的模型，放到真實影像上往往表現得很差。

二、三條縮小域落差的路

1. 真實場景重建：不要自己建模，直接把真實場景掃進來。目前的做法是以實拍影片重建出可互動的三維場景（例如 NuRec 函式庫與開源的 3DGUT 方法），這樣背景與材質本來就是真的，只有動態物件是合成的。
2. 視覺域轉換：用生成模型把模擬影像「翻譯」成看起來像真實拍攝的影像。Isaac Sim 的資料寫出器已能直接串接這類模型（如 Cosmos Transfer）。
3. 視覺域隨機化：反過來做——不去模仿真實，而是把模擬的材質、光線、背景隨機到極端誇張的程度，逼模型只能依賴物件的形狀而非外觀。這條路最簡單，也最常用。

三、合成與真實的混合策略

實務上最有效的通常不是二選一，而是混合：以大量合成資料做預訓練、再以少量真實資料做微調。這呼應第 32 章提過的一個重點——從預訓練權重微調，所需的真實資料量遠低於從零訓練。一個對校園專題務實的配方是：合成資料數千張用於預訓練，真實資料每類 200 至 500 張用於微調，並且測試集一律只用真實資料。

圖 33-2　資料從蒐集到部署的迭代流程

① 定義類別與蒐集規劃

↓

② 取得資料（真實拍攝 / 模擬生成 / 兩者混合）

↓

③ 標註（真實資料要訂規範並抽查；合成資料自動產生）

↓

④ 分割資料集（依地點而非隨機分割；測試集只用真實資料）

↓

⑤ 訓練（合成資料預訓練 → 真實資料微調）

↓

⑥ 評估（在真實測試集上計算精度；並在真實場域實測）

↓

⑦ 錯誤分析（哪些類別最常錯？什麼情況下錯？）

↓

⑧ 針對弱點補充資料 → 回到 ⑤ 迭代

↓

⑨ 部署最佳化（量化、加速）→ 邊緣裝置（33.8 節）

請特別注意第七與第八步：錯誤分析與針對性補資料，往往比盲目增加訓練輪數有效得多。合成資料在這一步特別有價值——當你發現模型在「逆光的樓梯」上最常錯，你可以在模擬裡專門生成一千張逆光樓梯，這在真實世界需要等天氣。

兩個值得知道的工具

MobilityGen：自動生成移動機器人的軌跡與對應的感測資料，適合用來產生 SLAM 與導航的訓練或評估資料集。

以視覺語言模型自動描述場景：模擬器算繪出影像後，由 VLM 產生文字描述作為標註——這把第 32 章的語意標註與本章的合成資料接了起來，也是第 34 章語言導航

資料的主要來源之一。

33.6　域隨機化與 sim-to-real 落差

這是本章最重要的一節。前面五節都在講「怎麼在模擬裡訓練」，這一節講「訓練完之後，為什麼搬到真機上會失敗，以及怎麼辦」。

一、落差從哪裡來

落差來源	具體內容	本書關聯
致動器動力學	韌體補償造成的虛擬慣量、電流環頻寬、扭矩飽和	本節
連桿慣量誤差	CAD 模型低估實際慣量（電線、外殼、感測器）	本節
摩擦與阻尼	減速機的庫倫摩擦與隨狀態改變的阻尼	本節
時間延遲	指令傳輸、濾波與控制週期造成的總延遲	第 25 章
關節偏差	組裝與校正誤差造成的零位偏移	第 10 章
感測器特性	雜訊模型、取樣時間、視角與遮擋	第 7–9 章
環境差異	地面摩擦、地形、負載、溫度與老化	第 19 章

表 33-4　sim-to-real 落差的七個來源

二、對策一：系統識別

第一條路是把模擬改對：量測真實機器人的反應，反推模擬應該用的參數。標準做法是把機

器人吊在空中（讓腳不接觸地面以排除接觸力的干擾），對每個關節送出掃頻訊號（頻率由低到高連續變化的正弦波），錄下編碼器的回應，再以最佳化反推參數。近期的研究指出，每台機器人只需要約 20 秒的空中掃頻資料就足夠，待識別的參數約為每關節四個（電樞慣量、黏滯阻尼、庫倫摩擦、關節偏差）加上一個全域的指令延遲。

以下用一個可完整重現的實驗示範這個流程。「真實機器人」是一個含庫倫摩擦與 7.5 毫秒指令延遲的關節模型；「URDF 模型」則只有連桿慣量，沒有阻尼、沒有摩擦、沒有延遲——這正是從 CAD 直接匯出的 URDF 的典型狀態。

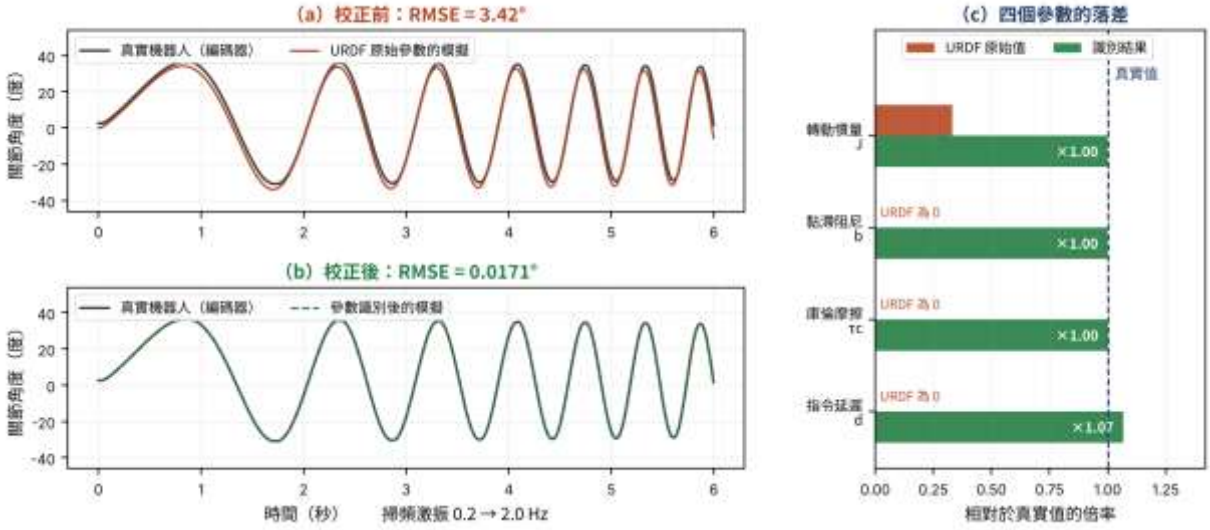


圖 33-3　系統識別：從 URDF 的猜測到擬合後的模擬

（a）是校正前。灰線是「真實機器人」編碼器錄到的角度，紅線是用 URDF 參數模擬出來的同一段動作。兩條線的形狀相似，但紅線明顯超前——這是延遲沒有被建模的後果；振幅也略大，因為模擬裡沒有摩擦與阻尼在消耗能量。整段的均方根誤差是 3.42 度。對一個行走策略而言，每個關節三度多的系統性偏差足以讓步態完全走樣。

（b）是校正後。綠色虛線與灰線完全重合，均方根誤差降到 0.0171 度，是校正前的兩百分之一。要強調的是：這不是把模型改複雜了，而是把同一個模型的參數調對了——方程式的形式從頭到尾沒有改變。

（c）說明參數到底差多少。紅柱是 URDF 的原始值相對於真實值的倍率，綠柱是識別結果。

轉動慣量上，URDF 只有真實值的 0.33 倍——少掉的三分之二來自韌體補償造成的虛擬慣量、以及 CAD 沒算進去的線材與外殼。黏滯阻尼、庫倫摩擦、指令延遲這三項，URDF 裡根本就是零。識別之後，慣量、阻尼、摩擦三項都還原到真實值的 1.00 倍；延遲因為是以 2 毫秒為間隔做格點搜尋，落在 8.0 毫秒（真值 7.5 毫秒），誤差一個格點。

> **一個實作上的坑：延遲不能用梯度法擬合**
>
> 指令延遲在離散時間的模擬裡是整數個時間步，因此殘差對延遲是階梯函數——在每個台階內部的梯度是零。把延遲丟進 least_squares 這類梯度法裡，它會停在起始值動都不動。
>
> 正確的做法是外層對延遲做整數格點搜尋，內層再以梯度法擬合連續參數。本節的實驗第一次做錯時，擬合出來的延遲是 0.0001 秒（真值 0.0075），而且慣量被高估了 2.3 倍去「補」那個沒被識別出來的延遲——軌跡看起來吻合，參數卻全錯。這是系統識別最典型的陷阱：誤差小不代表參數對，不同參數之間可能互相補償。

三、對策二：域隨機化

第二條路反過來：不去追求模擬正確，而是讓策略對參數不確定本身免疫。做法是在訓練時，每個環境都從一個分布裡隨機抽取動力學參數，逼策略學會一套「對任何合理參數都能用」的行為。

問題是：隨機的範圍該多寬？以下實驗比較三種設定。同一個關節追蹤任務，訓練三組控制器：（a）只在名目參數上訓練；（b）在名目值 ±10% 的範圍內隨機；（c）在慣量 0.5 至 2.0 倍、摩擦 0.3 至 2.0 倍、延遲 0 至 28 毫秒的寬範圍內隨機。然後把三組控制器放到同一張「真實參數」網格上評估。

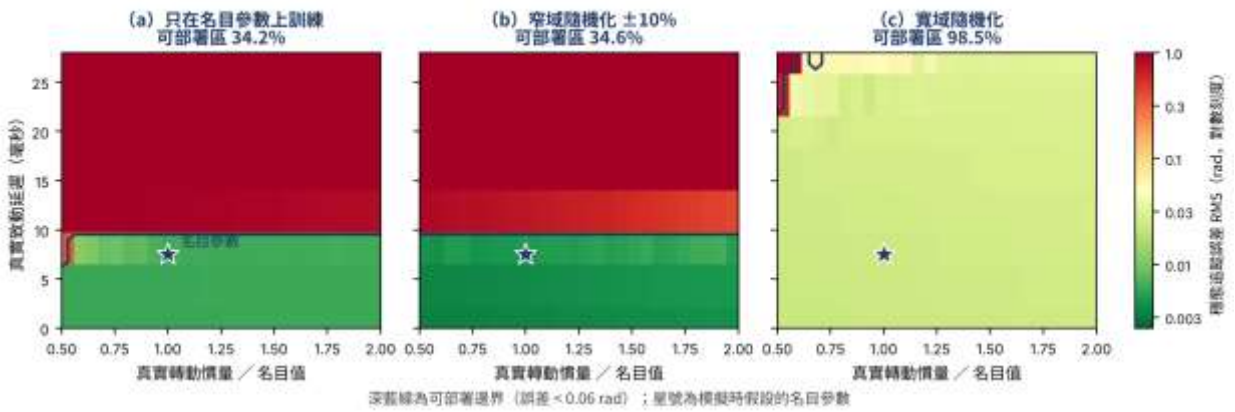


圖 33-4　域隨機化的範圍對部署成功率的影響

先看最明顯的結論：可部署區（深藍線包住的範圍）從（a）的 34.2%、（b）的 34.6%，跳到（c）的 98.5%。窄域隨機化幾乎沒有幫助——±10% 的範圍太窄，學到的策略和只在名目點訓練的策略差不多。這一點值得同學記住：域隨機化不是「加一點隨機就好」，範圍必須寬到涵蓋你對真實參數的實際不確定性。

第二個結論藏在顏色裡。（a）與（b）在可部署區內是深綠色，代表追蹤誤差極小；（c）全圖則是淡綠色，代表「到處都能用，但到處都不夠好」。量化來看：在名目參數這一點上，只在名目參數訓練的控制器誤差是 0.0061 弧度，寬域隨機化的控制器是 0.0245 弧度——四倍。這就是域隨機化的代價：穩健度不是免費的，它用名目效能去換。

第三個結論是失敗邊界的形狀。（a）與（b）的邊界幾乎是一條水平線，落在約 9.5 毫秒的延遲上，而且幾乎與轉動慣量無關。換句話說，在這個任務裡，慣量估錯兩倍不會讓策略失效，但延遲多幾毫秒就會。這個訊息對專題規劃很實用：與其花力氣去秤每個連桿的重量，不如先把控制迴路的延遲量出來。

四、兩條路怎麼選

	系統識別	域隨機化
做法	量測真機，把模擬改對	不改模擬，把策略訓得耐操
需要	能量測的真機與激振設備	對參數不確定範圍的估計

優點	名目效能高，可解釋	不需要真機即可訓練
缺點	參數可能互相補償；會隨磨損失效	名目效能下降；範圍難訂
適合	已有真機、要榨出最佳效能	真機尚未到位、或機群個體差異大

表 33-5　系統識別與域隨機化的比較

實務上兩者通常並用：先以系統識別把模擬的中心點校正到正確位置，再在這個正確的中心點周圍做適度的隨機化，涵蓋磨損、溫度與個體差異。這樣既不會像純域隨機化那樣犧牲名目效能，也不會像純系統識別那樣在零件磨損後失效。

33.7　怎麼上手、怎麼參加社群

這一節很實際：如果同學看完前面六節想動手，下一步該做什麼。

一、三條入門路徑

路徑	需要什麼	適合誰
雲端一鍵啟動	瀏覽器與帳號	沒有獨立顯示卡的同學
容器部署	具 CUDA 的 Linux 主機 + 容器環境	實驗室有工作站
原始碼安裝	同上，另需自行處理相依套件	要修改模擬器的人

表 33-6　三條入門路徑

對大多數同學，建議從雲端一鍵啟動開始。它把驅動程式、CUDA 版本、相依套件這些最容易卡住的環節都處理好了，讓你在第一天就能看到機器人在模擬裡動起來，而不是花三天在裝環境。等到流程熟悉、確定要長期投入，再轉到本地部署。

本地部署時有一個細節值得先知道：模擬器的畫面是透過 WebRTC 串流出來的，預設使用

49100 與 47998 這兩個連接埠。如果你在學校的網路環境下連不上畫面，十之八九是防火牆擋了這兩個埠，而不是軟體裝壞了。

二、一條建議的學習順序

1. 先跑通內建範例：不要改任何東西，確認你的環境能訓練出一個會走路的機器人。這一步的目的只是建立基準線。
2. 換機器人不換任務：把範例裡的機器人換成另一個內建型號，觀察同一套獎勵在不同形態上的表現差異。框架內建約十一種足式機器人形態，夠你比較很久。
3. 改獎勵：這是最能學到東西的一步。把速度追蹤的權重調高、把能耗懲罰調高，觀察步態怎麼改變。你會很快體會到獎勵設計是一門手藝。
4. 匯入自己的機器人：用匯入精靈把自己的 URDF 轉成模擬可用的格式，這時會開始遇到關節限制、慣量、摩擦設定的真實問題。
5. 接上 ROS 2：讓模擬器當成你導航堆疊的測試台（第八篇）。這一步不需要強化學習，但對做 SLAM 的同學價值最高。
6. 做系統識別與 sim-to-real：有真機之後，按 33.6 節的流程走一遍。

三、怎麼參加社群

機器人模擬是一個變動很快的領域，文件常常落後於程式碼。因此「知道去哪裡問」跟「知道怎麼做」幾乎一樣重要。

- 開發者論壇：官方的機器人開發者論壇是提問最有效的地方，工程師會直接回覆。提問時附上版本號、顯示卡型號與完整錯誤訊息，得到有用回覆的機率會高很多。
- GitHub：Isaac Sim 與 Isaac Lab 的程式碼在 GitHub 上公開。遇到行為與文件不符時，直接讀原始碼往往比搜尋快。也可以提交問題回報與修改——這是最實質的參與方式。
- 官方課程與電子報：免費的機器人基礎課程適合完全沒有背景的同學；機器人電子

報則是追蹤版本變動最省力的方式。

- 論文：技術報告與論文裡的數字（如 33.3 節引用的吞吐量）是規劃硬體採購最可靠的依據，比行銷資料準確得多。

> **給專題同學的三個提醒**
>
> 版本要鎖定。這個生態更新很快，不同版本之間的 API 會變。專題開始時就把版本號寫進文件，並且整個團隊用同一版。
>
> 不要在模擬裡追求完美。模擬的價值在於快速迭代，不在於像素級的真實。花三週把材質調得很漂亮，對策略的 sim-to-real 成功率幫助通常是零。
>
> 先想清楚要量什麼。訓練出一個會走路的機器人只是起點，專題的價值在於你量了什麼、比較了什麼（第 31 章）。

33.8　邊緣部署：Jetson、量化與蒸餾

訓練在雲端或工作站，執行卻在機器人身上。這一節處理這條「訓練完就部署」的一條龍最後一段：怎麼把模型塞進一台靠電池供電、巴掌大的運算模組裡。

一、問題的規模

視覺語言模型的運算需求通常使它們只能在雲端執行，而雲端執行又帶來第 25.1 節討論過的所有問題：依賴網路、延遲、頻寬與可靠度。對一台在地下停車場或廠房裡工作的機器人而言，「網路斷了就停住」不是可以接受的設計。

二、硬體的選擇

模組	AI 效能	記憶體與頻寬	功耗	典型用途
AGX Thor T5000	2,070 TFLOPS（FP4 稀疏）	128 GB，273 GB/s	40–130 W	機上跑大型模型
AGX Orin 64GB	275 TOPS	64 GB，204.8	15–60 W	SLAM + 多模型

	（INT8）	GB/s		並行
Orin NX 16GB	100 / 157 TOPS	16 GB，102.4 GB/s	10–40 W	巡檢機器人主流
Orin Nano 8GB	40 / 67 TOPS	8 GB，68 GB/s	7–25 W	教學與專題

表 33-7　邊緣運算模組的比較

看這張表時有兩個陷阱要避開。第一，Thor 標示的是 FP4 稀疏的 TFLOPS，Orin 系列標示的是 INT8 的 TOPS，兩者不能直接相除比較——數字差了七倍不代表效能差七倍。第二，也是更重要的：對同時跑 SLAM 與 AI 模型的機器人而言，決定能不能跑的往往不是算力那一欄，而是記憶體那一欄。案例 33-2 會說明這件事。

三、三大關鍵技術

量化：以較少的位元表示模型參數。目前邊緣端的視覺語言模型與視覺語言動作模型，預設採用 4 位元權重搭配 16 位元啟動值的組態；4 位元權重加 4 位元啟動值的格式能解鎖新一代硬體平台，是支援時的產品級升級選擇；8 位元權重加 8 位元啟動值則仍是成熟視覺與分割模型較安全的備選。

蒸餾：把大模型的能力轉移到小模型。有研究提出蒸餾策略，把大型前沿模型的複雜空間語意推理轉移到輕量的 40 億參數本地模型上，以在嵌入式 GPU 裝置上執行；實驗顯示，以約 500 條前沿模型的推理軌跡微調小模型，即能有效賦予導航能力，成功率達 34.5%，相較大型雲端模型的 39.7% 已相當接近。

記憶體管理：這是常被忽略卻極關鍵的一環。有研究在 Jetson Orin Nano 上同時執行 SLAM 與 VLM，發現以幾何為基礎的地圖管理策略會發生記憶體不足的失敗與執行停滯；改採語意分區的關鍵幀載入卸載機制後，不僅消除了這兩個問題、維持了定位穩定性，還使吞吐量提升並降低了 21.7% 的延遲。

四、公式二：量化的記憶體估算

模型量化後的記憶體需求約為：

記憶體 ≈ 參數量 × 每參數位元數 / 8　（位元組）

以 40 億參數的模型為例：16 位元精度需約 8 GB、8 位元約 4 GB、4 位元約 2 GB。這解釋了為何 4 位元量化能讓大模型擠進嵌入式平台——差距是四倍。對照表 33-7：一個 4 位元量化的 40 億參數模型約佔 2 GB，在 8 GB 的 Orin Nano 上還要與作業系統、SLAM 的地圖、影像緩衝區共用剩下的 6 GB，已經相當吃緊。

但要注意：實際記憶體用量還包含啟動值、中間結果與框架開銷，通常是上述估算的 1.2 至 2 倍。規劃時務必預留餘裕。

五、三種對策

1. 限制地圖規模：依語意分區只載入當前需要的區域。這呼應第 23 章 Atlas 的多地圖與第 15 章 Cartographer 的子地圖思想——把大問題切成可管理的小塊。
2. 選用量化後的小模型：不必為每個任務各裝一個大模型。一個量化過的通用模型，往往比三個未量化的專用模型更務實。
3. 錯開執行：AI 推論不需要每幀都跑。每秒一到兩次通常已足夠支撐語意層的決策，而 SLAM 的前端仍維持原本的頻率。

部署前的檢查清單

在目標硬體上量過記憶體峰值了嗎？（不是平均值，是峰值）

把 SLAM 與 AI 模型同時跑過了嗎？分開跑都正常不代表合起來正常。

量過端到端延遲了嗎？從影像進來到決策出去，不只是模型推論那一段。

斷網的情況測過了嗎？如果有任何雲端相依，這是必測項目。

長時間運行測過了嗎？記憶體洩漏與地圖成長要跑數小時才看得出來（第 19.6 節）。

案例

案例 33-1　沒有校正的模擬，訓練出來的策略會直接摔倒

這是一個把 33.6 節的系統識別講得最清楚的真實實驗。

研究團隊對多款足式機器人做了同一件事：把機器人吊在空中，對關節送出 20 到 60 秒的掃頻訊號，只用編碼器的資料，以演化式最佳化在 4,096 個平行環境中擬合動力學參數。待識別的參數包含每個關節的電樞慣量、黏滯阻尼、庫倫摩擦與零位偏差，加上一個全域的指令延遲，總共約四十九個。

關鍵的對照實驗是這樣的：用未經校正的 URDF 參數訓練出來的策略，放到真實的四足機器人上，連向前行走都失敗，機器人必須由吊車接住；而以識別後的參數訓練出來的策略則能正常行走。同一套演算法、同一組獎勵、同一個訓練流程，差別只在模擬參數對不對。

團隊同時量化了落差的來源，每一項都呼應表 33-4：韌體的補償行為造成約 8.1×10^{-3} kg·m² 的「虛擬慣量」，CAD 低估了大腿連桿的慣量，全域指令延遲約 7.5 毫秒，而關節零位偏差最大可達約 5 度。這些數字沒有一項能從 CAD 圖面讀出來。

校正帶來的不只是「能走」。同一套流程在不同機型上量到的運輸成本（CoT）從 1.86 降到 1.27，約三成的能耗改善；續航方面，一款四足機器人單次充電跑了 4.12 公里，另一款達 6.10 公里；一台 4.2 公斤的小型四足也能在 46 秒內爬完二十階、每階 18 公分的樓梯。

對同學的啟示有兩點。其一，約 20 秒的資料就能完成識別——這是一個學期專題完全負擔得起的實驗。其二，也是更重要的一點：團隊在識別之後並未使用動力學的域隨機化。這與 33.6 節圖 33-4 的結論一致——當你能把模擬校正到足夠準，就不必用名目效能去換穩健度。域隨機化是在你無法校正時的保險，不是預設選項。

案例 33-2　記憶體才是真正的瓶頸

這是許多同學在專題後期才會撞上的問題，提前認識能省下大量挫折。

情境是這樣的：同學在 Jetson 上成功跑起了 SLAM，也成功跑起了物件偵測模型，但一旦兩者同時執行，系統就變得極度緩慢，甚至直接崩潰。檢查處理器與圖形處理器的使用率，卻發現都沒有滿載——這讓人困惑。

真正的瓶頸是記憶體。相關研究對這個現象有清楚的量化：在 Jetson Orin Nano 上同時執行 SLAM 與視覺語言模型時，以幾何為基礎的地圖管理策略會遭遇記憶體不足的失敗與執行停滯；而改採語意分區的關鍵幀載入卸載機制後，不僅消除了這兩個問題並維持定位穩定性，還提升了吞吐量並降低了 21.7% 的延遲。

問題的根源是：SLAM 的地圖會隨探索持續成長（第 13.5 節的記憶體估算、第 14 章的粒子數乘地圖大小），而 AI 模型本身就佔用大量記憶體。兩者相爭，系統必然崩潰。

解法的巧妙之處在於「語意分區」：機器人不需要隨時記得整棟樓的細節，只需載入當前所在區域的地圖。這個思想與第 23 章 Atlas 的多地圖、第 15 章 Cartographer 的子地圖一脈相承——把大問題切成可管理的小塊。

對專題的具體建議：在規劃階段就用公式二估算記憶體需求，並優先考慮限制地圖規模的機制。等到系統崩潰才處理，往往需要大幅重構。

案例 33-3　把大模型裝進機器人：蒸餾的威力

這個案例回應同學最可能有的疑問：大模型跑不動，是不是就只能放棄？

問題的背景很清楚：視覺語言模型在機器人領域已成為強大的工具，能以語言脈絡理解環境，催化了物件目標導航這類開放詞彙任務；但它們的運算量通常使其只能在雲端執行。

研究團隊的做法是蒸餾：先以大型前沿模型建立一套基於場景圖的高水準管線，在標準的開放詞彙物件導航基準上達到 39.7% 的成功率；接著把這套推理能力轉移到僅 40 億參數的本地模型上，以便在嵌入式 GPU 裝置上執行。

關鍵的結果是：僅以約 500 條前沿模型的推理軌跡微調小模型，就能有效賦予它導航能力，

成功率達到 34.5%，大幅縮小了與大型雲端模型之間的差距。

五百條資料——這個數字值得細細品味。相較於從零訓練所需的數十萬筆標註，這是極小的成本。而換來的是「能在機器人上跑」這個質變：不需要網路、沒有雲端延遲、資料不外流。

這個案例的方法論啟示，與第 25 章 NanoSLAM 的案例完全一致：效能瓶頸往往不在算力不足，而在方法未經最佳化。當你覺得「這個技術太重、跑不動」時，先問的不該是「要不要放棄」，而是「有沒有更聰明的做法」。

實作：從模擬訓練到邊緣部署

本章的實作分三個階段，難度遞增。階段一不需要任何特殊硬體，階段三需要一台嵌入式運算模組。

階段一　平行度的三張圖（不需要機器人）

重現本章圖 33-1 的三個量測。任務可以自己訂，最簡單的是本章用的單關節追蹤，也可以用任何你熟悉的向量化環境。

1. 量吞吐量：固定策略，對 N = 1, 2, 4, …, 4096 各跑一批，記錄牆鐘時間並換算成每秒環境步數。畫在對數座標上，標出飽和點。
2. 量梯度雜訊：固定策略，對每個 N 重複抽樣四十次，計算成本估計的標準差。驗證它是否依 1/√N 下降（乘上 √N 看是否接近常數）。
3. 量訓練穩定度：固定牆鐘時間預算，對不同 N 各跑五次獨立訓練，畫出中位數與四分位距。重點不是誰的中位數高，而是誰的四分位距窄。

這個階段的產出是一張「你的硬體的平行度曲線」。它會告訴你：在你的機器上，環境數開到多少之後就不再划算。這是後續所有訓練實驗的規劃依據。

階段二　系統識別與域隨機化（需要一個有編碼器的關節）

最低需求是一個帶編碼器的伺服馬達或減速機模組，能送出位置指令並讀回實際角度。一個機械手臂的單關節就夠了。

1. 錄掃頻資料：讓關節不接觸任何負載，送出 0.2 至 2 Hz 的掃頻位置指令，以盡可能高的頻率記錄指令與編碼器讀值，錄 20 至 60 秒。
2. 建模型：寫出含轉動慣量、黏滯阻尼、庫倫摩擦與指令延遲的離散時間模型。先用 CAD 的慣量值、其餘設為零，模擬一次並與實測比較——這就是你的圖 33-3（a）。
3. 做識別：外層對延遲做整數格點搜尋，內層以最小平方法擬合連續參數。記得驗證參數是否互相補償：換一段不同的激振訊號重新驗證，如果換了訊號參數就跑掉，表示識別還不夠穩。
4. 量代價：用識別後的參數訓練一組控制器，再用寬範圍域隨機化訓練另一組，在真實關節上比較兩者的追蹤誤差。你應該會看到與圖 33-4 相同的現象：隨機化的那組比較耐操，但名目精度較差。

階段三　邊緣部署（需要嵌入式運算模組）

1. 建立基準線：在模組上單獨跑 SLAM，記錄記憶體峰值、延遲與更新率。再單獨跑你的 AI 模型，同樣記錄三項。
2. 合併執行：兩者同時跑，再量一次三項。比較「合併後」與「分開時的總和」，差額就是競爭造成的損失。
3. 量化：把模型量化到 8 位元與 4 位元，各量一次記憶體與精度。用公式二的估算值對照實測值，看看框架開銷佔多少。
4. 套用對策：依 33.8 節的三種對策逐一嘗試，每次只改一項並重新量測，建立一張「對策 × 效果」的表。

這個階段的產出是一份部署報告，內容是你自己量到的數字。它比任何規格表都有說服力，也是本書技術路線最實際的收尾。

延伸閱讀

- Isaac Lab 技術報告（2025）。（GPU 原生平行模擬的架構與吞吐量數據，33.3 節的依據）
- Isaac Sim 與 Isaac Lab 的 GitHub 專案與官方文件。（33.2、33.7 節的實作基礎）
- 足式機器人的系統化 sim-to-real 轉移研究（2025）。（33.6 節與案例 33-1 的來源）
- 學習式足式運動控制的綜述論文。（本章 33.1 節的背景，含各家方法的比較）
- OpenUSD 規格與機器人結構描述文件。（33.2 節場景與機器人描述格式的參考）
- NVIDIA Jetson 開發者文件與推論最佳化指南。（33.8 節邊緣部署的技術基礎）
- 第 25 章的邊緣運算、第 31 章的評估方法、第 32 章的資料蒐集與標註。（本章與全書其他章節的直接銜接點）

本章重點

- 強化學習的機器人策略需要十億等級的環境互動，在真機上相當於約 231 天的連續運行，因此必須先在模擬中訓練。模擬提供時間加速、零成本失敗、完全可知的狀態、可重複的實驗與可生成的環境；但它解的是你寫進去的方程式，不是真實物理。
- Isaac Sim 是模擬器與場景環境（PhysX 物理、RTX 算繪、OpenUSD 場景），Isaac Lab 是建構其上的機器人學習框架（任務、觀測、獎勵、訓練流程）。兩者於 2025 年 8 月開放一般取用並將擴充層開源，但底層的 Omniverse Kit 並未全部開源，也不接受外部提交。
- GPU 原生模擬的關鍵是狀態以張量常駐 GPU，整個訓練迴圈不需往返 CPU。實測顯示吞吐量在 $N \le 32$ 時近乎線性（每秒 3.5 萬步增至 113 萬步），其後飽和於每秒約 2,285 萬步；但環境數增加 4,096 倍時，每批次的等待時間只增加 6.3 倍。

- 平行度的真正價值是降低梯度雜訊。實測的成本估計標準誤精確地依 1/√N 下降（乘上 √N 後十一個點都落在 0.32 至 0.43）。在相同的 22 秒預算下，N=4 的成功率中位數只有 35.3% 且四分位距橫跨 15.8–85.6%，N=64 達 98.4%，N=1024 達 99.2% 且四分位距僅 98.3–99.4%——平行度不足時，訓練結果的變異大到無法比較。
- 感測器仿真可分四個家族：本體感測（高）、相機（中高）、測距（中）、接觸與觸覺（中低）。相機的分割、深度、法向量是算繪的副產品，這是合成資料的技術基礎；而光達的鏡面反射、運動畸變與感測器之間的時間差，是最常被忽略的落差來源。
- 合成資料的誘惑是標註免費且零誤差，陷阱是域落差。三條縮小落差的路是真實場景重建、視覺域轉換與視覺域隨機化；實務上最有效的是合成資料預訓練加少量真實資料微調，且測試集一律只用真實資料。
- sim-to-real 落差有七個來源，其中致動器動力學、連桿慣量誤差與時間延遲最常被低估。系統識別實驗顯示：URDF 的轉動慣量只有真實值的 0.33 倍，而阻尼、摩擦、延遲三項根本是零；校正後模擬與真實的均方根誤差由 3.42 度降到 0.0171 度，約兩百分之一。
- 系統識別有一個典型陷阱：延遲在離散時間下是階梯函數，用梯度法會停在起始值不動，而慣量會被高估去補償它——軌跡吻合但參數全錯。正確做法是外層對延遲做格點搜尋。
- 域隨機化的範圍必須寬到涵蓋真實的不確定性：±10% 的窄範圍幾乎無效（可部署區 34.6%，與不隨機化的 34.2% 相當），寬範圍則達 98.5%。但代價是名目點的追蹤誤差從 0.0061 弧度增為 0.0245 弧度，約四倍——穩健度是用名目效能換來的。
- 實測的失敗邊界幾乎只由延遲決定（約 9.5 毫秒），與轉動慣量幾乎無關。這說明規劃時應優先量測控制迴路的延遲，而非精算每個連桿的質量。
- 兩種對策通常並用：先以系統識別把模擬的中心點校正正確，再在其周圍做適度隨機化以涵蓋磨損、溫度與個體差異。案例 33-1 顯示，校正做得夠好時可以完全不用動力學的域隨機化。

- 邊緣部署的三大技術為量化（4 位元權重是目前邊緣視覺語言模型的預設）、蒸餾（約 500 條推理軌跡微調 40 億參數模型即可達 34.5%，接近雲端大模型的 39.7%）、以及記憶體管理（語意分區的關鍵幀載入卸載能消除記憶體不足並降低 21.7% 延遲）。
- 選擇邊緣模組時，決定能不能跑的往往是記憶體那一欄而非算力那一欄；且不同世代的標示單位不同（FP4 稀疏的 TFLOPS 與 INT8 的 TOPS 不可直接相比）。實際記憶體用量約為公式二估算值的 1.2 至 2 倍。

承先啟後

本章把全書的技術從「能跑」推到「能學」：第 32 章讓地圖有了意義，本章則讓機器人有了在模擬中取得技能的途徑，並把訓練好的模型一路送到機器人身上的運算模組裡。

但還有最後一塊拼圖沒有補上。本章訓練出來的是「怎麼走」的策略，第 32 章建立的是「這裡是什麼」的地圖——中間缺的是「為什麼要去那裡」。當使用者說「去會議室把投影機關掉」，機器人需要把一句話拆成一連串可執行的子目標，需要在沒見過的環境裡推理「會議室通常在哪」，也需要在找不到目標時知道該往哪裡探索。

這就是第 34 章的主題：視覺語言模型、語言導航與具身智慧。它也是全書的最後一章——我們會把第 1 章到第 34 章的線索收攏起來，用一次完整的任務說明這三十四章各自在哪一個環節發揮作用。

台灣自編大學教科書

@prof

《地圖建構與同步定位》

Simultaneous Localization and Mapping, SLAM

第 34 章

VLM、語言導航與具身智慧

編者：李世淵（機器人叫獸）

助教：李天宇、李宇晴

大學智慧機器人學程　/　高中 AI 機器人社團課程

本章在全書的位置

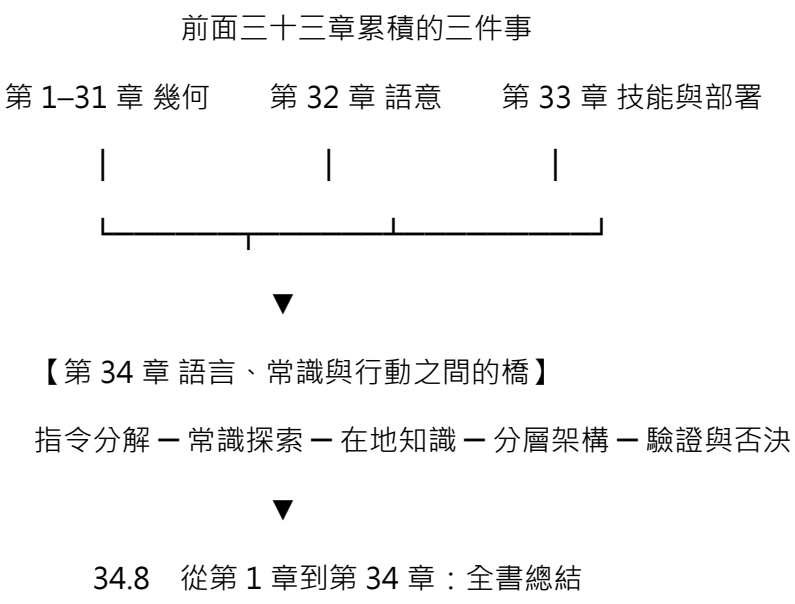
這是全書的最後一章，也是把前面三十三章收攏起來的一章。

到目前為止，我們有了三樣東西：第 1 至 31 章給了機器人一張準確的幾何地圖，讓它知道「我在哪、環境長什麼形狀」；第 32 章讓地圖有了意義，讓它知道「那是一扇門、那是一個人」；第 33 章讓它能在模擬中學會技能，並把模型送進機身上的運算模組。

還缺最後一樣：目的。當使用者說「去會議室把投影機關掉」，機器人必須把一句話變成一連串可執行的子目標，必須在沒去過的樓層推理「會議室大概在哪一側」，也必須在找不到目標時知道該往哪裡繼續找。這就是本章的主題——語言、常識與行動之間的橋。

本章同時承擔全書的總結。34.8 節會用一次完整的任務，把第 1 章到第 34 章的每一篇放回它該在的位置，說明為什麼這九篇缺一不可。

圖 34-0　第 34 章的連結地圖



學習目標

本章從「一句話怎麼變成座標」開始，依序處理常識先驗、在地知識庫、分層架構與驗證機制，再誠實地檢視目前視覺語言模型在導航上的真實水準，最後以全書總結作結。本章有三張以實驗算出來的圖，分別回答三個問題：常識值多少、常識錯了會怎樣、以及為什麼一定

要有驗證層。讀完本章，同學應能：

1. 說明自然語言指令轉為導航目標的各個階段，並指出每一階段可能的失敗模式。
2. 解釋常識先驗如何縮短物件目標導航的搜尋成本，並能量化其效益。
3. 說明一個指錯方向的先驗會造成什麼代價，並據以判斷何時不該相信語意層。
4. 設計一個在地空間知識庫，並說明檢索增強生成相較於微調模型的三個優點。
5. 說明依時間尺度分層的系統架構，並指出每一層的更新頻率與職責。
6. 以連乘模型估算多步驟任務的整體成功率，並說明驗證與重試為何能大幅改善它。
7. 說明目前視覺語言模型在具身導航上與人類表現的差距，並能解讀相關基準的結果。
8. 把全書九篇的內容對應到一次完整任務的各個環節。

核心概念

名詞	意義	關聯章節
具身智慧	智慧體透過身體與環境互動而產生的智慧	34.1、第 33 章
語言導航	依自然語言指令在未知環境中導航	34.2、34.6
物件目標導航	給定一個物件名稱，找到它的位置	34.3
常識先驗	「某類物件通常在某類空間」的統計知識	34.3
語意探索	以語意先驗決定下一個探索方向	34.3、第 16 章
檢索增強生成	推論前先從知識庫檢索相關資訊	34.4

場景圖	以節點與關係描述場景的結構化表示	34.1、第 32 章
分層架構	依時間尺度把系統切成數層，各司其職	34.5、第 3 章
否決機制	下層可推翻上層的決策以保障安全	34.5、34.7
視覺語言動作模型	直接由影像與語言輸出動作的模型	34.6、第 33 章

表 34-1　本章的核心名詞

34.1　從「知道那是什麼」到「知道該做什麼」

第 32 章結束時，機器人手上有一張標了語意的地圖。本節說明，從那裡到「能執行一句話」，中間還差什麼。

一、三個層次，再看一次

第 32 章用三個層次描述機器人的智慧：幾何層回答「我在哪、環境什麼形狀」，語意層回答「我看到了什麼」，任務層回答「下一步該怎麼做」。前兩層本書已經處理完畢，本章處理第三層。

任務層與前兩層有一個本質差異：前兩層的輸入是感測器，輸出是對世界的描述；任務層的輸入是人的意圖，輸出是行動。描述可以求準確，意圖卻常常是模糊的——「把會議室的燈關掉」沒說是哪一間會議室、也沒說燈的開關在哪裡。任務層的工作有一大半是在補上這些沒說出口的東西。

二、一句話裡藏了多少假設

以「去會議室把投影機關掉」為例，機器人必須自行補上的資訊至少有：

- 哪一間會議室？（若有多間，可能要依使用者所在位置或行事曆推斷）

- 會議室在哪？（需要在地的空間知識，34.4 節）
- 投影機長什麼樣子？（需要語意層的辨識能力，第 32 章）
- 投影機在會議室的哪裡？（需要常識：通常在天花板或講桌上）
- 「關掉」要怎麼做？（按遙控器？按機身按鈕？還是發紅外線訊號？）
- 怎麼確認真的關掉了？（需要驗證，34.5 節）
- 關不掉怎麼辦？（需要降級與回報，34.7 節）

這七項裡面，只有第三項是第 32 章處理過的。其餘六項分別需要在地知識、常識推理、動作知識與驗證機制——這正是本章要補齊的部分。

三、為什麼場景圖比語意地圖更有用

第 32 章提過場景圖：不只記錄「這裡有一張椅子」，還記錄「椅子在桌子旁邊、桌子在會議室裡」的關係結構。到了任務層，這個差異就變得關鍵。

理由是：語言裡的空間關係是相對的。「桌上的杯子」、「門旁邊的滅火器」、「靠窗那排的第三個座位」——這些在平面的語意地圖上無法表達，因為語意地圖只記錄「某座標有某類物件」。要回答「桌上的杯子」，系統必須知道哪個杯子的支撐關係指向哪張桌子。

一個區分：導航的三種目標形式

點目標：直接給定座標（「去（12.4, 8.1）」）。這是第 1 至 31 章處理的形式，也是所有上層最終都必須轉成的形式。

物件目標：給定一個物件類別（「去找飲水機」）。目標位置未知，需要搜尋——這是 34.3 節的主題。

指令目標：給定一段自然語言（「沿著走廊走到底，右轉進第二間教室」）。需要逐步理解並執行——這是 34.6 節評估的對象，也是目前最困難的一種。

34.2　語言指令怎麼變成導航目標

本節把這條轉換鏈拆開來看，因為每一個環節都是一個可能的失敗點。

一、五個階段

階段	輸入	輸出	典型失敗
① 意圖解析	自然語言	結構化的任務描述	指代不明、動詞歧義
② 地點定位	任務描述	語意地圖上的區域	地點名稱不在知識庫中
③ 目標定位	區域 + 物件類別	三維座標	物件不在原處、辨識錯誤
④ 路徑規劃	三維座標	可行路徑	目標不可達、門關著
⑤ 執行與確認	路徑	完成或回報	抵達了但沒確認、中途被打斷

表 34-2　從語言到行動的五個階段

值得注意的是：第④階段之後的一切，本書前三十一章已經完全處理過了。本章與第 32 章新增的，是第①到③階段。換句話說，語言導航不是取代 SLAM，而是在 SLAM 的前面接上一段轉換。

二、輸出要結構化，不要自由文字

一個實作上的重要原則：上層送給下層的，應該是結構化的資料（目標座標、物件清單、動作代號），而不是自由文字。

理由很實際。自由文字沒有辦法可靠地解析，也沒有辦法驗證。如果語言模型回答「往左前方那間看起來像茶水間的房間走」，下游系統要怎麼把它變成座標？而如果它回答的是「目標區域：茶水間，信心 0.72；候選座標 （23.3, 13.9）」，下游就能查詢地圖驗證該座標是否可達、是否在禁區內、是否與代價地圖衝突。

這也是為什麼「讓語言模型直接輸出速度指令」在工程上是糟糕的設計。它跳過了所有可以驗證的中間表示，把一個可以被檢查的系統變成一個不可檢查的黑盒子。

三、不確定性要往下傳

第 11 章花了整章講機率與不確定性，這個觀念在任務層同樣適用，而且常常被忽略。語言模型通常會用同樣肯定的語氣說出「會議室在三樓東側」，無論它是從知識庫查到的、還是猜的。

務實的做法是：讓上層輸出的每個目標都帶一個信心值與一個來源標記（查自知識庫 / 由常識推得 / 純粹猜測）。下層依這個標記決定要多謹慎——查自知識庫的目標可以直接前往，純粹猜測的目標應該先派出一趟低成本的探索再決定。

34.3　常識先驗與語意探索

「洗手台通常在廁所或茶水間」——這類知識無法從幾何地圖推導，卻能大幅改變機器人的搜尋效率。本節用一個可完整重現的實驗來量化它值多少。

一、實驗設定

場景是一層系館樓層：中央走廊加上兩側十二個房間，房間有六種類型（教室、實驗室、辦公室、茶水間、會議室、廁所、儲藏室）。機器人從入口出發，要找到一個指定的物件（飲水機、滅火器、投影機、急救箱、梯子、影印機之一）。

關鍵的設定是：兩種策略拿到的是同一張地圖。這不是「有地圖」對「沒地圖」的比較——那不公平，而是同一張先驗地圖之下，搜尋順序的比較：

- 幾何順序：每次前往最近的未搜尋房間。這是不使用任何語意資訊的基準。
- 語意順序：依「這類物件出現在這類房間的機率」由高到低排序，機率相同時取較近的房間。

物件依一組真實的條件機率分布隨機放置（例如飲水機有 55% 的機率在茶水間、30% 在走

廊、15% 在會議室）。機器人進入房間後執行來回掃掠（掃掠長度依第 16 章的覆蓋規劃估算），當物件進入 3.5 公尺的感測範圍且視線不被牆擋住時視為找到。共跑 600 次隨機試驗。

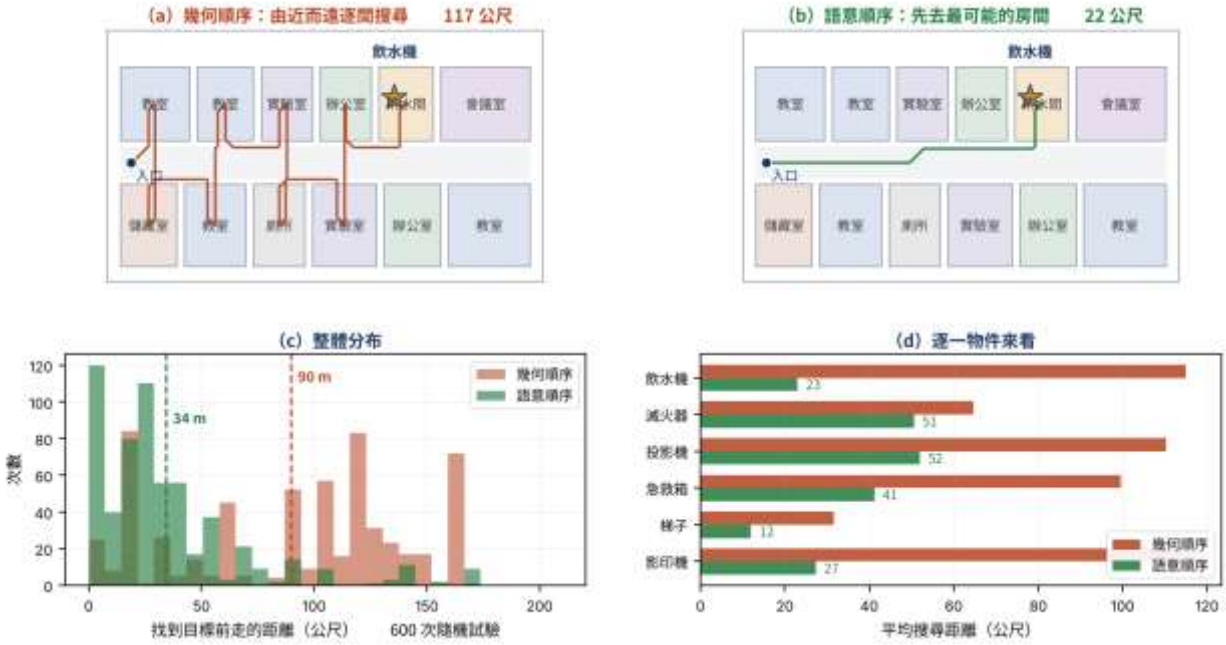


圖 34-1　同一張地圖，兩種搜尋順序

（a）與（b）畫的是同一次試驗：飲水機落在茶水間。幾何順序從入口旁最近的教室開始，一間一間往東推進，走了 117 公尺才抵達茶水間。語意順序直接前往茶水間，只走了 22 公尺。圖上畫的是房間之間的移動路徑，房內的來回掃掠沒有畫出來，但距離有計入。

（c）是 600 次試驗的整體分布。幾何順序的平均是 90 公尺，語意順序是 34 公尺，平均縮短 61.7%；若看中位數則是 102.5 公尺對 26.5 公尺，縮短 74.2%。兩個分布的形狀也不同：幾何順序呈現明顯的多峰，每一個峰對應「物件落在第幾間被搜尋的房間」；語意順序則大量集中在 30 公尺以內。

（d）逐一物件來看，效益差異很大。梯子從 32 公尺降到 12 公尺、影印機從 118 公尺降到 27 公尺，因為它們的分布高度集中（梯子有 70% 在儲藏室、影印機有 65% 在辦公室）。滅火器只從 65 公尺降到 51 公尺，因為它有 60% 的機率在走廊上——而走廊是機器人無論如何都會經過的地方，語意先驗在這裡幫不上什麼忙。

這個差異本身就是一個有用的結論：常識先驗的價值，取決於該物件的分布有多集中。對於

「到處都有」的物件（滅火器、垃圾桶、消防栓），花力氣建立語意推理的報酬率很低；對於「只在特定地方」的物件（梯子、影印機、特定儀器），報酬率就很高。

一個常被跳過的數字

在這 600 次試驗裡，語意順序有 12.2% 的情況反而比幾何順序走得更遠。

原因是：機率最高的房間不一定是這一次物件真正所在的房間。當飲水機那 15% 的機率落在會議室時，先衝去茶水間的機器人就白跑了一趟。

這提醒我們：語意先驗改善的是「期望值」，不是「每一次」。在評估任何語意導航系統時，只看平均值會低估它的變異——這也是第 31 章一再強調的評估原則。

二、先驗錯了會怎樣

前面的實驗假設機器人的常識是對的。但視覺語言模型的常識是從網路資料學來的，不一定符合你的建築。「影印機在辦公室」在多數場合成立，但如果這棟樓把影印機統一放在走廊的開放區域呢？

下面的實驗掃描先驗的正確率 α：以機率 α 使用真實的分布，否則把房間類型的排序隨機打亂。另外再加一個極端情況——先驗「反著指」，也就是真實機率越高的房間，排序越往後。

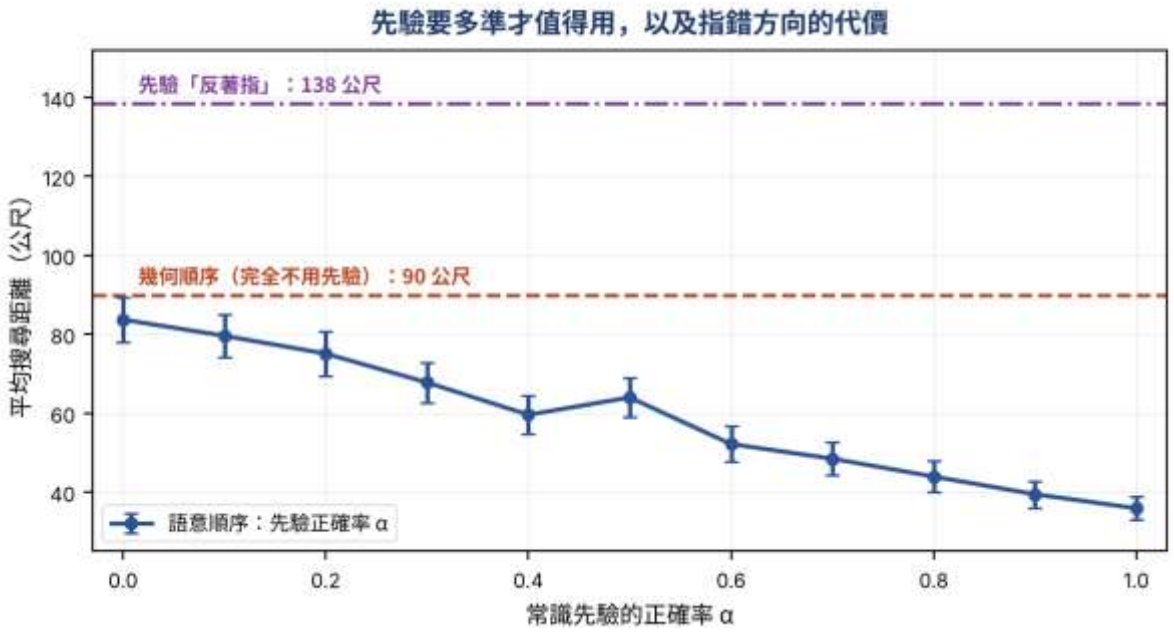


圖 34-2　先驗要多準才值得用，以及指錯方向的代價

藍線是語意順序隨 α 的變化，誤差棒是 95% 信賴區間。從 α=0 的 84 公尺一路降到 α=1 的 36 公尺，大致呈線性。這條線性關係本身是個好消息：常識先驗不需要完美才有用，正確率每提升一成，就省下約五公尺。

紅色虛線是完全不用先驗的幾何順序（90 公尺）。注意藍線即使在 α=0 時也略低於它——一個完全隨機的搜尋順序，和「由近而遠」的順序差不多好。這有點反直覺，但道理不難懂：物件是隨機放置的，「先去近的房間」並不會提高找到它的機率，只是先省下一點移動距離而已。

真正的風險在紫色點線：當先驗反著指時，平均搜尋距離是 138 公尺，比完全不用先驗還糟 54%。這就是語意層最危險的失效模式——它不是沒幫上忙，而是主動把機器人帶往錯誤的方向，而且它會用同樣肯定的語氣這麼做。

對系統設計的啟示很明確：語意層的建議必須可被推翻。合理的做法是設一個預算——若依語意順序搜尋了前兩個建議仍未找到，就退回幾何順序，不再相信先驗。這正是 34.5 節「否決機制」的具體形式。

34.4　RAG：讓機器人讀懂在地的空間知識

上一節的常識先驗處理的是「一般而言」的知識。本節處理另一半：這一棟樓特有的知識。

一、問題與解法

這裡有一個實務問題：視覺語言模型再聰明，也不知道「這所學校的教務處在哪一棟哪一樓」——這是特定機構的知識，不在模型的訓練資料中。

解法是檢索增強生成（RAG）：建立一份在地的空間知識庫，當機器人收到指令時，先從知識庫檢索相關資訊，再交給模型推理。模型負責語言理解與推理，知識庫負責事實。

圖 34-3　校園空間知識庫的範例結構

【樓層拓樸】

一樓：校門口 → 穿堂 → 教務處 → 學務處 → 樓梯 → 電梯

二樓：電梯 → 走廊 → 電腦教室 → 會議室

【地點屬性】

教務處：一樓、穿堂東側、開放時間 08:00–17:00、門為玻璃自動門

智慧機器人教室：三樓、入口 → 工作桌 → 機器狗測試區 → 光達建圖區 → 充電區

【通行規則】

機器人不可進入：實驗室內部、樓梯（除非為爬樓梯平台）

需減速區域：穿堂、教室門口、轉角

【常見查詢對應】

「辦公室」→ 教務處、學務處　　「充電」→ 各樓層充電站

二、RAG 的三個優點

1. 可維護：校園配置改變時，只需更新知識庫，不需重新訓練模型。一間教室改了用途，改一行文字就好。

2. 可驗證：機器人的回答可以追溯到知識庫的哪一條，便於除錯與問責。這與 34.7 節的可解釋性問題直接相關。
3. 低成本：相較於微調大模型，建立知識庫的成本低得多，也更適合教學專題——一份結構化的文字檔就能起步。

三、三個實用的查詢範例

- 「從入口到機器狗測試區怎麼走？」——RAG 檢索到智慧機器人教室的內部拓樸，模型據此生成路線描述，再轉為導航目標序列。
- 「如果前方有學生坐在地上，機器人應該如何繞路？」——結合即時感知（偵測到人）與知識庫的通行規則（需減速、禮讓），產生繞行策略。
- 「掃地機器人如何規劃覆蓋整間教室？」——結合地圖（第 13 章）與覆蓋規劃（第 16 章），這是語意層與幾何層協作的典型例子。

四、務實的架構建議

對專題實作而言，建議的分工是：RAG 知識庫用結構化的文字檔或簡單資料庫即可（不必一開始就上向量資料庫）；語言模型負責語言理解與視覺判斷；而所有涉及安全的決策（避障、速度限制）仍由傳統的幾何系統把關。這是下一節反覆強調的原則。

> **知識庫與常識先驗的分工**
>
> 一個容易混淆的地方：34.3 節的常識先驗與本節的知識庫，都是「模型之外的知識」，但兩者的性質完全不同。
>
> 常識先驗是統計性的、跨場域通用的（「梯子通常在儲藏室」），它回答的是「該先去哪裡找」。知識庫是確定性的、場域專屬的（「儲藏室在一樓西側」），它回答的是「那裡在哪裡」。
>
> 一個好的系統兩者都要：用知識庫把能查到的查清楚，查不到的才交給常識先驗去猜——而且要記得標明這是猜的（34.2 節）。

34.5　分層架構與驗證機制

本節說明如何把前面各節的元件組裝成一個不會因為上層出錯而失控的系統。

一、依時間尺度分層

圖 34-4　語言導航系統的分層架構

【任務層】秒級以上　語言模型 Agent

- 理解自然語言指令、查詢知識庫（34.2、34.4 節）
- 分解任務、選擇探索方向、判讀異常（34.3 節）

↓ 輸出：語意目標（去哪、找什麼）+ 信心值 + 來源

【語意層】0.1–1 秒　視覺 AI 模型

- 物件偵測、分割、深度估計（第 32 章）
- 語意地圖的建立與查詢

↓ 輸出：幾何目標座標、動態物體標註

【幾何層】10–30 Hz　SLAM + 導航（第 1–31 章）

- 定位建圖、路徑規劃、代價地圖

↓ 輸出：速度指令

【控制層】100 Hz 以上　底盤 / 飛控（第 33 章）

- 運動控制、即時避障、安全停止

【安全原則】上層可以出錯，下層必須守住安全底線

這個分層與第 3 章的混合式架構、第 15 章 Cartographer 的雙子系統、第 18 章 LOAM 的雙層設計、第 26 章無人機的降級架構，是同一個系統設計模式在 AI 時代的延伸——依時間尺度分層、各層各司其職。同學至此應該能深刻體會：好的架構原則具有跨時代的普遍性。

二、整合的四個實務要點

1. 非同步設計：AI 推論耗時，絕不能阻塞導航迴路。應以非同步方式呼叫，並在等待期間讓機器人維持安全行為（繼續執行上一個目標或停下）。

2. 明確的介面：AI 層輸出的應該是結構化的資料，而非自由文字（34.2 節）。這讓下游系統能可靠地解析與驗證。

3. 可否決機制：幾何層應能否決 AI 的決策。若 AI 說「往前走」但代價地圖顯示前方是障礙，幾何層必須勝出。圖 34-2 的紫色點線說明了沒有否決機制的代價。

4. 降級路徑：AI 服務不可用時（斷線、逾時、模型崩潰），系統應退回純幾何導航，而非停擺——這正是第 26.2 節「失效逐級降級」的原則。

三、為什麼多步驟任務一定要有驗證

前面講的「可否決機制」聽起來像是一個安全上的保險。但它其實還有一個更基本的作用：沒有它，多步驟任務根本不可能有堪用的成功率。下面用一個簡單的模型說明。

假設一個任務有 n 個步驟，每一步的成功率都是 p，且各步驟獨立。那麼整體成功率是 p 的 n 次方——這是公式一。這個連乘關係的破壞力，遠比直覺來得大。

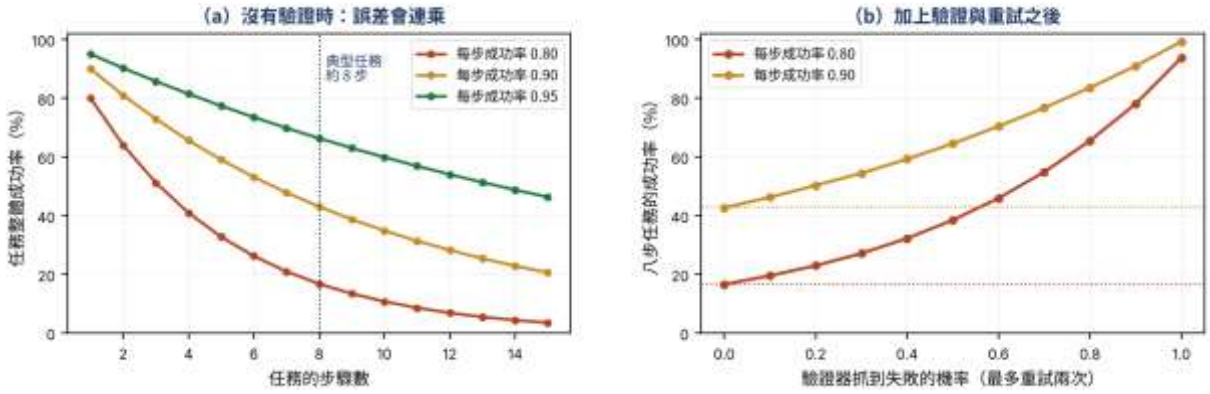


圖 34-5　多步驟任務的成功率，以及驗證與重試的效果

（a）畫的就是連乘的後果。一個八步的任務（34.2 節的五個階段再加上開門、對位、回報，八步是很保守的估計），即使每一步都有 90% 的成功率，整體成功率只有 43.0%；每步 80% 的話更只剩 16.8%。要讓八步任務達到 66.3%，每一步必須做到 95%。

這解釋了一個常見的困惑：為什麼機器人的每個子系統看起來都不錯，整體卻老是失敗？不是哪一個環節特別糟，而是連乘本身就很殘酷。它也解釋了為什麼「把任務拆得更細」不一定是好事——拆得越細，步驟越多，連乘的懲罰越重。

（b）是對策。在每個步驟後面加上一個驗證：若驗證器判定這一步失敗，就重試（最多兩次）。橫軸是驗證器抓到失敗的機率，點線是沒有驗證時的水準。以每步 80% 的系統為例，整體成功率從 16.6% 提升到驗證率 0.5 時的 38.5%、驗證率 1.0 時的 93.7%。每步 90% 的系統則從 42.7% 提升到 99.3%。

請注意這個改善的幅度：一個完美的驗證器，把一個「十次有八次失敗」的系統變成「十次有九次成功」，而底層每一步的能力完全沒有改變。這是本章最重要的工程結論之一——對多步驟任務而言，投資在「知道自己做錯了」上面，報酬率遠高於投資在「做得更好」上面。

更何況驗證通常比執行容易得多。「走到會議室門口」需要完整的導航堆疊，但「確認自己是不是在會議室門口」可能只需要讀一次門牌、或比對一次當前位姿與目標位姿。這種「驗證比執行簡單」的不對稱，是分層架構能夠成立的根本原因。

四、公式一：多步驟任務的成功率

設任務有 n 個步驟，第 i 步的成功率為 p_i，各步獨立，無驗證與重試，則：

$$P_{\text{任務}} = p_1 \times p_2 \times \dots \times p_n$$

當各步成功率相同時簡化為 $P = p^n$。若加入驗證（偵測率 r）與最多 k 次重試，單一步驟的有效成功率提升為：

$$p_{\text{eff}} = p + (1 - p) \cdot \Sigma_{j=1..k} \left(r(1 - p)\right)^{j - 1} \cdot r \cdot p$$

其中 $r \cdot p$ 是「被抓到並重試成功」的機率。當 $r \to 1$ 且 k 夠大時，$p_{\text{eff}} \to 1$，整體成功率趨近 1；當 $r \to 0$ 時 $p_{\text{eff}} \to p$，退回（a）的情況。圖 34-5(b) 的曲線就是這條式子在 n=8、k=2 下的數值結果。

34.6　現況：視覺語言模型在導航上到底有多行

前面五節描述了一個理想的架構。本節誠實地檢視：以 2025 至 2026 年的技術水準，這個

架構裡最上面那一層做得如何。

一、一個具身導航的基準

2025 年底發表的一個具身導航基準提供了相當清楚的答案。它的設計是：給模型一張真實場景的照片、一段文字指令，以及一個載體類型（人、四足機器人、輪型機器人、腳踏車），要模型在影像上畫出應該走的軌跡。資料集包含 1,000 個情境與超過 3,000 條專家軌跡，評分方式結合了軌跡的動態時間校正距離、終點誤差，以及依語意分割計算的「載體不該走的地方」的懲罰。

八個當代的視覺語言模型被評估。結果如下：

方法	分數
人類專家	75.40
只預測路徑（直接告訴它目標在哪）	56.55
已知目標的直線基準	51.89
完整軌跡預測（最佳模型）	34.38
只預測目標點	29.65

表 34-3　具身導航基準的評估結果（分數越高越好）

二、三個應該被記住的數字

第一，最佳模型的 34.38 分與人類專家的 75.40 分之間，差距是 41.02 分。這不是一個可以用「再過半年就追上」來形容的差距。2026 年的視覺語言模型在具身導航上仍遠不及人類。

第二，也是最值得玩味的一個：一條「已知目標的直線」拿到 51.89 分，遠高於最佳模型完整軌跡的 34.38 分。一條完全不看環境、不避障、不繞路的直線，竟然勝過當代最強模型的完整規劃。這說明模型的失分主要不在路徑形狀，而在它根本沒搞清楚目標在哪。

第三點佐證了這個判斷：當目標被直接告知時，同一個模型只預測路徑能拿到 56.55 分，反

過來只預測目標點則掉到 29.65 分。也就是說，模型「會走」但「不會認」——把語言裡的目標對應到影像中正確的位置，才是目前真正的瓶頸。

三、這對系統設計意味著什麼

這三個數字直接支持本章 34.4 與 34.5 節的設計原則。既然模型最弱的環節是「目標定位」，那麼系統就應該盡可能不讓它做這件事——能從知識庫查到的地點，絕不讓模型猜（34.4 節）；必須猜的時候，要標明是猜的並設定搜尋預算（34.2、34.3 節）；而猜錯的代價，要由驗證與否決機制吸收（34.5 節）。

反過來說，模型相對擅長的事情應該交給它：理解語言、判讀畫面、生成報告、在已知目標的情況下描述路線。這就是一個務實的分工——不是問「語言模型能不能做導航」，而是問「導航的哪些部分適合交給它」。

> **另一個現況：真實部署仍然稀少**
>
> 2026 年的一篇綜述指出，語言導航的研究進展「大多仍侷限於軟體層面的真實轉模擬設定」，對模擬轉真實與真實世界部署的研究相對有限，而「邊緣部署的實用方案也相當稀缺」。
>
> 這與第 33 章的結論互相呼應：模擬裡的成績與真機上的表現是兩回事。對同學而言，這其實是機會——一個在真實校園裡跑得起來的語言導航系統，即使功能簡單，在目前也具有實質的研究價值。

34.7　風險、限制與未來發展

在收尾之前，必須談三件事：風險、限制、以及方向。

一、四大風險

1. 幻覺：AI 可能「看到」不存在的東西或做出錯誤推理，而且它通常不會表達不確定。若機器人完全信任 AI，可能撞牆或走錯路。圖 34-2 的紫色點線正是這個風險的量化

形式。對策是保留幾何層的驗證與否決權（34.5 節）。

2. 誤判的系統性：AI 的錯誤往往不是隨機的，而是有系統的偏差——例如在特定光線下總是把某類物件認錯。這比隨機錯誤更危險，因為它會持續發生，而且重試也救不了（34.5 節的驗證模型假設重試是獨立的，系統性錯誤違反這個假設）。對策是充分的場域測試與錯誤分析（第 32 章、第 33.5 節）。

3. 延遲：大模型推論耗時，若放在即時控制迴路中會造成災難。對策是嚴格的分層架構——AI 只做秒級以上的決策（34.5 節）。

4. 安全性：包含資安（模型或知識庫被竄改）、隱私（機器人拍攝的影像可能含個人資訊）、以及可問責性（出事時能否追溯決策依據）。這些不只是技術問題，也是倫理與法規問題。

二、當前的四個限制

- 資料需求：訓練或微調模型需要標註資料，而標註成本高昂。這是專題與小型團隊的主要障礙——也是為何蒸餾、少量樣本微調與合成資料（第 33.5、33.8 節）如此受重視。

- 泛化的落差：模型在實驗室表現良好，換個場域就下降。第 33.5 節的域落差討論同樣適用於語意層。

- 可解釋性：AI 為什麼做出這個判斷？當它出錯時，往往難以診斷。這與傳統幾何方法「每一步都可追溯」形成鮮明對比，也是 34.4 節主張用知識庫而非微調的理由之一。

- 運算與能源：如第 33.8 節所述，即使經過量化與蒸餾，AI 仍是機器人上最耗資源的元件之一。

三、未來的四個方向

1. SLAM 與基礎模型的深度整合：不再是「SLAM 加上 AI 模組」，而是統一的架構——由基礎模型同時處理感知、建圖與規劃。

2. 世界模型：讓機器人不只理解「現在是什麼」，還能預測「接下來會發生什麼」。這對動態環境中的規劃至關重要，也與第 33 章的模擬訓練直接相通——一個夠好的世界模型，本身就是一個可以在裡面訓練的模擬器。
3. 機器人基礎模型：跨平台的通用代理——同一套模型能部署到輪型、四足、人形、飛行等不同平台。這將大幅降低機器人開發的門檻。
4. 終身學習：機器人在部署後持續學習與適應，而非停留在出廠時的能力。第 25.6 節提過的裝置端自監督學習是這個方向的早期嘗試，而第 19.6 節的地圖維護則是它最務實的雛形。

34.8　從第 1 章到第 34 章：一台機器人的完整旅程

作為全書的最後一節，讓我們用一次完整的任務，把三十四章的內容串起來。

一、一次任務，九篇知識

情境：台灣南部一所科技大學的服務機器人接到指令——「請去三樓的智慧機器人教室，檢查充電區的設備狀態，並回報異常」。

1. 理解指令：模型解析出目標地點與任務內容（34.2 節）。輸出是結構化的任務描述，不是自由文字。
2. 查詢知識庫：RAG 檢索出「智慧機器人教室在三樓、內部拓樸為入口→工作桌→機器狗測試區→光達建圖區→充電區」（34.4 節）。這一步讓機器人不必用猜的。
3. 規劃路線：在既有地圖上規劃從當前位置到三樓的路徑，包含搭電梯（第 3 章、第 29 章）。
4. 移動與定位：光達 SLAM 或視覺慣性 SLAM 持續估計位姿（第 13 至 23 章），在既有地圖中定位（第 11 章、第 29 章），IMU 在轉彎與震動時提供支撐（第 7 章）。
5. 通過電梯口：看到人工標記，取得絕對位姿校正並確認樓層（第 4 章、第 24 章）。
6. 避開行人：偵測到人，一方面剔除其特徵以免干擾定位，一方面觸發減速禮讓（第

24 章、第 32 章）。

7. 抵達充電區：依語意地圖找到目標，以視覺伺服完成精確對位（第 24 章）。若語意地圖上找不到，則依常識先驗決定往哪裡找，並在兩次嘗試後退回幾何搜尋（34.3 節）。
8. 檢查與判讀：拍攝設備影像，AI 判斷是否異常，並把發現標上三維座標（第 31 章、第 32 章）。所有移動與操作的技能，來自第 33 章的模擬訓練。
9. 驗證每一步：每個階段完成後確認結果，失敗則重試（34.5 節）。沒有這一步，上面這八步的整體成功率會低到不堪用。
10. 回報：以自然語言生成報告，附上位置與影像證據（34.2 節）。

這一趟任務中，本書的每一篇都有貢獻：第一篇的觀念、第二篇的感測器、第三篇的數學、第四至六篇的 SLAM 系統、第七篇的邊緣運算、第八篇的實作工具、第九篇的 AI 語意、模擬訓練與語言導航。

而最關鍵的是：這九篇缺一不可。少了幾何層，機器人會撞牆；少了語意層，它聽不懂指令；少了模擬訓練，它學不會動作；少了工具鏈，一切都無法運行。這正是本書從第一頁到最後一頁想傳達的訊息——SLAM 不是單一技術，而是一整套互相支撐的知識體系。

二、一條貫穿全書的主線

本書從第 1 章的「機器人沒有地圖，也不知道自己在哪裡」開始，走過感測器、數學、2D 與 3D 系統、視覺方法、微型與群體機器人、完整實作，最後來到 AI 語意導航、模擬訓練與語言理解。

如果要說這三十四章有一個貫穿的主線，那就是：理解限制，才能突破限制。

我們反覆討論每種感測器的弱點（第二篇）、每種演算法的失敗模式（第四至六篇）、每個平台的資源約束（第七篇）、每個系統的除錯順序（第八篇）、以及每個模型的落差來源（第九篇）。這不是悲觀，而是因為工程的本質就是在限制中求解。

AI 的加入沒有改變這個本質。語言模型讓機器人「聽得懂、想得通」，但「走得穩、不撞人」仍然要靠前面三十一章的幾何與控制。本章 34.6 節的數字說得很清楚：在具身導航上，當代最強的模型仍遠不及人類，而一條已知目標的直線甚至勝過它的完整規劃。兩者缺一不可——而且可預見的未來也將如此。

三、給同學的最後幾句話

本書的兩個場域——大學智慧機器人學程與高中 AI 機器人社團課程——代表了兩種學習階段，但共享同一個信念：SLAM 不是遙不可及的尖端科技。它的核心版本可以跑在一顆手機等級的處理器上，它的原理可以用紙筆推演，它的成果就在你家客廳的掃地機器人裡默默運轉。

本書裡的每一張實驗圖，都是用一台普通的電腦算出來的。第 15 章的迴路閉合、第 17 章的配準、第 18 章的運動補償、第 19 章的巡檢路線、第 33 章的平行模擬與參數識別、以及本章的語意搜尋——沒有一張需要昂貴的設備。需要的只是把問題寫清楚、把實驗做乾淨、把數字誠實地看進去。

願同學帶著這本書打下的基礎，去建造屬於自己的機器人——無論它是巡檢校園的機器狗、掃描古蹟的無人機，還是一台只會沿線走的小車。每一台能自己找到路的機器人，背後都是同一套人類花了四十年才建立起來的智慧。

現在，輪到你了。

案例

案例 34-1　一句話、八個步驟：為什麼驗證比能力更重要

這個案例把 34.5 節的數字放回一個具體的情境裡。

假設同學做了一台校園送件機器人，任務是「把這份文件送到三樓教務處」。拆解之後有八個步驟：理解指令、查詢地點、規劃路線、搭電梯、走到門口、辨識門牌、進入、交付並確

認。

假設每一步都做到 80% 的成功率——這聽起來不差，每個子系統十次有八次成功。但整體成功率只有 0.8 的八次方，也就是 16.8%。十次任務裡有八次會在某個環節出錯。

團隊的直覺通常是去改善最弱的那一環。假設花了一整個學期，把每一步從 80% 提升到 90%——這是相當可觀的工程投入。結果呢？整體成功率從 16.8% 提升到 43.0%。進步很大，但十次仍有將近六次失敗。

另一條路是：每一步都不改，但在每一步後面加一個檢查。「電梯按了嗎？」比對樓層感測器。「到門口了嗎？」比對當前位姿與目標位姿。「門牌對嗎？」拍一張照辨識文字。檢查失敗就重做這一步，最多兩次。

以 34.5 節的模型計算：若檢查能抓到一半的失敗，整體成功率從 16.6% 提升到 38.5%；若檢查接近完美，則提升到 93.7%。而底層每一步的能力，一點都沒有改變。

這個案例要傳達的，不只是一個數字上的技巧。它是一個關於工程優先順序的判斷：當一個多步驟系統的整體表現不佳時，先問「它知不知道自己做錯了」，通常比先問「怎麼讓它做得更好」更有效。這與第 31 章的除錯順序、第 26 章的降級設計，是同一個思路。

案例 34-2　當機器人的常識指錯方向

這個案例處理一個同學在專題中很可能遇到、卻很少被事先提醒的失效模式。

情境：同學為巡檢機器人加上了語意搜尋的能力。機器人要找「影印機」時，會先問語言模型「影印機通常在哪裡」，得到「辦公室」的答案，於是優先搜尋所有辦公室。在測試的那棟樓，這個策略表現極好——搜尋距離縮短了七成。

接著他們把系統搬到另一棟樓。那棟樓為了省電與管理方便，把所有影印機集中放在每層樓電梯旁的開放區域，辦公室裡一台都沒有。結果機器人每次都先把所有辦公室跑一遍，然後才在「最不可能」的走廊上找到目標。

圖 34-2 量化了這個情況。當先驗完全反著指時，平均搜尋距離是 138 公尺，而完全不用先驗的幾何順序是 90 公尺——語意層讓系統退步了 54%。更糟的是，系統不會知道自己錯了：它每一次都「合理地」先去辦公室，每一次都合理地失敗。

值得注意的是中間那段。圖 34-2 的藍線在 α=0（先驗完全隨機）時是 84 公尺，略優於幾何順序的 90 公尺。也就是說，一個無知的先驗並不可怕，可怕的是一個有把握但錯誤的先驗。這正是語言模型最容易產生的那一種——它很少說「我不知道」。

對策有三個層次。最低限度是設搜尋預算：依語意建議搜尋前兩個地點，未果就退回幾何順序。進一步是把在地的觀察寫回知識庫：第一次在走廊找到影印機之後，這個事實應該被記下來，而不是下次再用通用常識猜一遍（34.4 節）。最徹底的做法是讓先驗本身可被學習——這正是第 33 章終身學習方向要處理的問題。

案例 34-3　一條直線，勝過最強的模型

這是一個適合放在全書最後的案例，因為它同時說明了 AI 的價值與它的邊界。

2025 年底的一個具身導航基準，要求模型在真實場景的照片上畫出機器人應該走的軌跡。評估的對象包含八個當代最強的視覺語言模型，評分同時考慮軌跡的形狀、終點的準確度，以及「這個載體不該走的地方」的懲罰。

結果中最令人意外的一項是：一條「已知目標位置的直線」得到 51.89 分，而最佳模型的完整軌跡預測只有 34.38 分。一條不避障、不繞路、完全不看環境的直線，贏過了當代最強模型的完整規劃。

第一反應可能是嘲笑模型。但仔細想，這個結果說的不是「模型很笨」，而是「評分裡絕大部分的分數來自目標的位置」。補充的實驗證實了這一點：同一個模型，在被直接告知目標位置後只預測路徑，分數躍升到 56.55；反過來只預測目標點，則掉到 29.65。

換句話說，模型「會走」但「不會認」。它能理解「沿著走廊往前、避開那堆箱子」這樣的

空間描述，但無法可靠地判斷「使用者說的那間會議室，是畫面裡的哪一扇門」。

這個診斷對系統設計極有價值，因為它指出了該把工程力氣花在哪裡。既然目標定位是瓶頸，那麼能查知識庫就不要猜（34.4 節）、能用人工標記就不要純靠視覺（第 4 章、第 24 章）、必須猜的時候要標明信心並設預算（34.2、34.3 節）。

同時它也說明了 AI 的真正價值在哪裡。模型拿到的 56.55 分（已知目標、只預測路徑）是高於直線基準的 51.89 分的——也就是說，在目標明確的前提下，模型確實能做出比直線更好的規劃。這就是本章 34.6 節的結論：不要問「語言模型能不能做導航」，而要問「導航的哪些部分適合交給它」。

最後值得補一句。人類專家在這個基準上的分數是 75.40，而最佳模型是 34.38——差距 41.02 分。這本書寫到最後一頁，技術仍在快速前進；但在可預見的未來，那套人類花了四十年建立起來的幾何方法，仍然是機器人能不能安全走到目的地的根本保證。

實作：校園語意導航系統

這是全書的最終實作，分四個階段，可依課程層次選擇進行到哪一階段。它同時用上第 32、33、34 三章的內容。

階段一　校園物件辨識資料集（高中適用）

1. 定義類別：與同學討論並決定五到十個校園常見物件（建議：教室門、白板、講桌、椅子、樓梯、電梯、走廊、垃圾桶、消防栓、飲水機、路障）。
2. 視角規劃：決定拍攝高度與角度，務必模擬機器人的實際視角（30 至 50 公分高），而非站著拍——這是本實作最重要的細節。
3. 分組蒐集：每組負責二至三類，每類拍攝至少 200 張，涵蓋不同角度、距離、光線與遮蔽情況，並刻意包含困難樣本。
4. 標註：先訂定標註規範（半掩的門算不算？框到哪裡？），再進行標註，並互相抽

查至少 10%。

5. 分割資料集：依「拍攝地點」而非隨機分割，避免評估結果虛高。
6. 訓練模型，在測試集上評估，並針對表現最差的類別做錯誤分析。

階段二　語意搜尋與常識先驗（本章核心）

這個階段不需要真實機器人，在一張地圖上就能完成，是本章三張圖的重現。

1. 建立樓層的語意地圖：把你們系館的平面圖數位化，標出每個房間的類型與門的位置。
2. 建立物件分布：調查五到八個物件在各類房間的實際出現頻率。這一步本身就是一個有價值的田野調查。
3. 實作兩種搜尋順序：幾何順序與語意順序，在隨機放置的物件上各跑數百次，比較搜尋距離的分布。這是你們的圖 34-1。
4. 掃描先驗品質：刻意把先驗打亂或反轉，量測搜尋距離如何變化，找出「先驗要多準才值得用」的交叉點。這是你們的圖 34-2。
5. 設計退場機制：實作「搜尋前兩個語意建議未果則退回幾何順序」的策略，量測它在先驗錯誤時能挽回多少。

階段三　在地知識庫與語言介面

1. 建立知識庫：依圖 34-3 的結構，寫出你們系館的樓層拓樸、地點屬性、通行規則與常見查詢對應。用純文字檔即可。
2. 實作檢索：給定一句指令，從知識庫檢索相關條目。先用關鍵字比對就好，不必一開始就上向量檢索。
3. 接上語言模型：讓模型根據檢索結果輸出結構化的目標（區域名稱、座標、信心值、來源標記）。
4. 測試失敗情況：故意問知識庫裡沒有的地點，確認系統會標明「這是猜的」而不是

假裝知道。

階段四　驗證層與端到端整合

1. 量測每一步的成功率：把你們的任務拆成 n 步，每一步單獨測 30 次以上，得到各步的 p_i。
2. 預測整體成功率：用公式一算出理論值，再跑 30 次完整任務，比較預測與實測。差異本身就是很好的討論題材。
3. 加上驗證：為每一步設計一個檢查，量測檢查的偵測率 r（故意製造失敗，看檢查抓到幾次）。
4. 量測改善：加上驗證與重試之後重測整體成功率，與公式一的預測比較。這是你們的圖 34-5。
5. 撰寫報告：報告的核心不是「我們做出來了」，而是「我們量到了什麼、哪裡與預測不符、為什麼」。這是本書從第 1 章到第 34 章一貫的要求。

延伸閱讀

- 具身導航的視覺語言模型評估基準（2025）。（34.6 節與案例 34-3 的來源，含完整的評分方法）
- 大模型增強的視覺語言導航綜述（2026）。（34.6 節關於真實部署現況的依據）
- 開放詞彙零樣本物件導航的相關研究。（34.3 節語意探索的理論基礎）
- 第 32 章的語意地圖與場景圖、第 33 章的模擬訓練與邊緣部署。（本章的直接前置，三章構成第九篇）
- 第 3 章的混合式架構、第 26 章的降級設計、第 31 章的評估與除錯方法。（34.5 節分層架構與驗證機制的思想來源）
- 第 11 章的機率與不確定性。（34.2 節「不確定性要往下傳」的理論基礎）

本章重點

- 任務層與感知層的本質差異在於：感知的輸入是感測器、輸出是對世界的描述；任務層的輸入是人的意圖、輸出是行動。意圖常是模糊的，任務層的工作有一大半是補上沒說出口的假設。
- 語言到行動可分五階段：意圖解析、地點定位、目標定位、路徑規劃、執行與確認。第四階段之後本書前三十一章已完全處理，語言導航是在 SLAM 前面接一段轉換，不是取代它。
- 上層送給下層的必須是結構化資料（座標、清單、信心值、來源標記），不是自由文字——自由文字無法解析也無法驗證。讓語言模型直接輸出速度指令，是把可檢查的系統變成不可檢查的黑盒子。
- 常識先驗的效益可觀：同一張地圖之下，語意順序的平均搜尋距離是 34 公尺，幾何順序是 90 公尺，縮短 61.7%（中位數縮短 74.2%）。但有 12.2% 的試驗語意順序反而更遠——它改善的是期望值，不是每一次。
- 效益取決於物件分布的集中程度：梯子（70% 在儲藏室）縮短 62%、影印機（65% 在辦公室）縮短 77%，但滅火器（60% 在走廊）只縮短 22%。對「到處都有」的物件，建立語意推理的報酬率很低。
- 先驗的正確率 α 與搜尋距離大致呈線性（α=0 時 84 公尺，α=1 時 36 公尺）。完全隨機的先驗（84 公尺）並不比不用先驗（90 公尺）差；真正危險的是反著指的先驗——138 公尺，比不用先驗還糟 54%。語言模型最容易產生的正是這種「有把握但錯誤」的先驗。
- RAG 以在地知識庫補足模型沒有的機構知識，三個優點是可維護、可驗證、低成本。常識先驗（統計性、跨場域、回答「先去哪找」）與知識庫（確定性、場域專屬、回答「那裡在哪」）分工不同，好的系統兩者都要。
- 分層架構依時間尺度切分：任務層（秒級）、語意層（0.1–1 秒）、幾何層（10–30

Hz）、控制層（100 Hz 以上）。四個實務要點是非同步設計、明確介面、可否決機制、降級路徑。安全原則是「上層可以出錯，下層必須守住底線」。

- 多步驟任務的成功率是連乘的：八步任務在每步 90% 時只有 43.0%，每步 80% 時只剩 16.8%。這解釋了為什麼每個子系統都不錯、整體卻老是失敗。
- 驗證與重試的效果遠大於提升單步能力：每步 80% 的八步任務，加上偵測率 0.5 的驗證（最多重試兩次）從 16.6% 升到 38.5%，偵測率接近 1 時升到 93.7%——底層能力完全沒變。驗證通常比執行容易得多，這種不對稱是分層架構能成立的根本原因。
- 2025 年底的具身導航基準顯示：最佳視覺語言模型得 34.38 分，人類專家 75.40 分，差距 41.02 分。更值得注意的是「已知目標的直線」得 51.89 分，勝過模型的完整軌跡；而同一模型在已知目標時只預測路徑可得 56.55 分，只預測目標點則掉到 29.65 分。模型「會走」但「不會認」——目標定位才是當前真正的瓶頸。
- 四大風險為幻覺、誤判的系統性、延遲、安全性。其中系統性誤判最危險，因為它違反「重試是獨立的」這個假設，重試救不了它。
- 全書的主線是「理解限制，才能突破限制」。AI 讓機器人聽得懂、想得通，但走得穩、不撞人仍要靠前三十一章的幾何與控制。在可預見的未來，兩者缺一不可。

全書結語

第九篇到此完成，全書也到此完成。

從第 32 章讓地圖有了意義、第 33 章讓機器人學會動作、到本章讓它聽懂了人話，這三章把前面三十一章累積的幾何基礎推向了應用的前沿。但請記得本章 34.6 節那幾個數字：最前沿的模型在具身導航上仍然遠不及人類，而一條已知目標的直線甚至勝過它的完整規劃。

這不是要澆冷水，而是要把位置看清楚。你們手上這套從第 1 章學到第 31 章的幾何與機率方法，不是即將被 AI 取代的舊技術，而是任何語言模型都必須站上去的地面。

三十四章，九篇，走到這裡。接下來要建造什麼，由你們決定。